

Thesis

Jason Leff

2025-09-19

Setup

```
rm(list=ls())
data <- read.csv('data_with_pre_data.csv')
set.seed(67)
```

Initializing Functions

```
heatmap <- function(ConfMatrix, color, title) { #Unused heatmap function
  cm_table <- as.data.frame(ConfMatrix$table)
  ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
    geom_tile(color = "white") +
    geom_text(aes(label = Freq), color = "black", size = 4) +
    scale_fill_gradient(low = "white", high = color) +
    theme_minimal() +
    labs(title = paste0("Confusion Matrix Heatmap (", title, ")"),
         x = "Actual Class", y = "Predicted Class")
}
```

```
within_one_class_pct <- function(ConfusionMatrix) { #Unused ordinal statistic calculator
  classes <- rownames(ConfusionMatrix)
  n_class <- length(classes)
  idx <- seq_len(n_class)
  total <- sum(ConfusionMatrix)
  count_valid <- 0
  for (i in idx) {
    valid_pred <- i
    if (i == 1) {valid_pred <- c(valid_pred, 2)}
    else if (i == n_class) {valid_pred <- c(valid_pred, n_class - 1)}
    else {valid_pred <- c(valid_pred, i - 1, i + 1)}
    count_valid <- count_valid + sum(ConfusionMatrix[i, valid_pred])
  }
  count_valid / total
}
```

```
macro_auc <- function(y_true, prob_matrix) { #Used to calculate AUC for each model
  classes <- levels(y_true)
```

```

aucs <- numeric(length(classes))
for (i in seq_along(classes)) {
  cls <- classes[i]
  y_binary <- as.numeric(y_true == cls)
  p <- prob_matrix[, cls]

  roc_obj <- try(roc(y_binary, p, quiet = TRUE), silent = TRUE)

  if (inherits(roc_obj, "try-error") || is.null(roc_obj$auc)) {
    aucs[i] <- NA_real_
  } else {
    aucs[i] <- as.numeric(roc_obj$auc)
  }
}
mean(aucs, na.rm = TRUE)
}

```

Normalize to Per Minute Data

```

normalize_stats <- function(df) {
  nba_start <- which(names(df) == "NBA_MIN") #split the data
  nba_end <- which(names(df) == "PRE_G")

  for (i in (nba_start+1):(nba_end-1)) {
    colname <- names(df)[i]
    if (!grepl("PCT", colname, ignore.case = TRUE) && is.numeric(df[[colname]])) { #divide total column
      df[[colname]] <- round(as.numeric(df[[colname]]) / as.numeric(df[["NBA_MIN"]]), 3)
    }
  }
  pre_start <- which(names(df) == "PRE_MP")
  for (i in (pre_start+1):ncol(df)) { #repeat for college
    colname <- names(df)[i]
    if (!grepl("PCT", colname, ignore.case = TRUE) && is.numeric(df[[colname]])) {
      df[[colname]] <- round(as.numeric(df[[colname]]) / as.numeric(df[["PRE_MP"]]), 3)
    }
  }
  return(df)
}
per_min_data <- normalize_stats(data)

```

PRE-NBA FT% and 3P% vs NBA 3P% filtered for 25 attempts

```

data_filtered2 <- data %>%
  filter(PRE_3PA >= 25 & PRE_FTA >= 25 & NBA_FG3A >= 25)

clean_data <- na.omit(data_filtered2[c("PLAYER", "NBA_FG3_PCT", "PRE_3P_PCT", "PRE_FT_PCT", "PRE_COL_or_...
model <- lm(NBA_FG3_PCT ~ PRE_3P_PCT + PRE_FT_PCT, data = clean_data)
summary(model)

```

```
##
## Call:
## lm(formula = NBA_FG3_PCT ~ PRE_3P_PCT + PRE_FT_PCT, data = clean_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.252344 -0.019491  0.008559  0.033728  0.095787
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.10796    0.01583   6.822 1.49e-11 ***
## PRE_3P_PCT     0.22357    0.03315   6.743 2.51e-11 ***
## PRE_FT_PCT     0.19565    0.02277   8.593 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.04865 on 1085 degrees of freedom
## Multiple R-squared:  0.1633, Adjusted R-squared:  0.1618
## F-statistic: 105.9 on 2 and 1085 DF,  p-value: < 2.2e-16
```

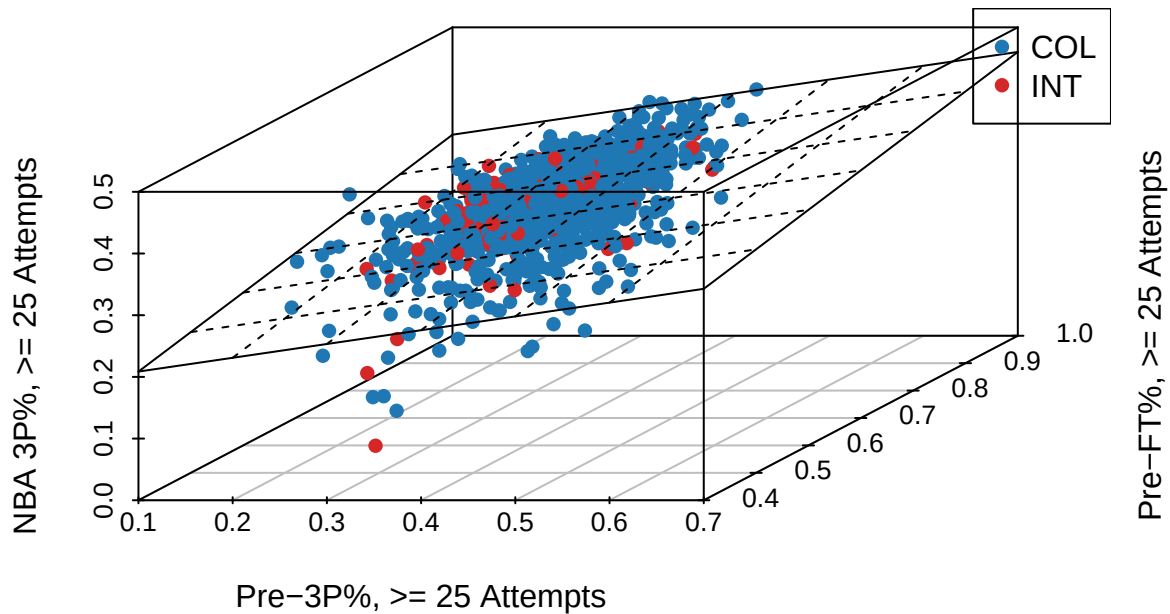
```
color_factor <- as.factor(clean_data$PRE_COL_or_INT)

my_colors <- c(
  "COL" = "#1f77b4", # blue
  "INT" = "#d62728"  # red
)

point_colors <- my_colors[color_factor]
s3d <- scatterplot3d(
  clean_data$PRE_3P_PCT,
  clean_data$PRE_FT_PCT,
  clean_data$NBA_FG3_PCT,
  color = point_colors,
  pch = 16,
  xlab = "Pre-3P%, >= 25 Attempts",
  ylab = "Pre-FT%, >= 25 Attempts",
  zlab = "NBA 3P%, >= 25 Attempts",
  main = "Pre-NBA 3P% and FT% vs. NBA 3P%, minimum 25 attempts"
)

s3d$plane3d(model, lty.box = "solid")
legend(
  "topright",
  legend = levels(color_factor),
  col = my_colors,
  pch = 16
)
```

Pre-NBA 3P% and FT% vs. NBA 3P%, minimum 25 attempts



Clustering

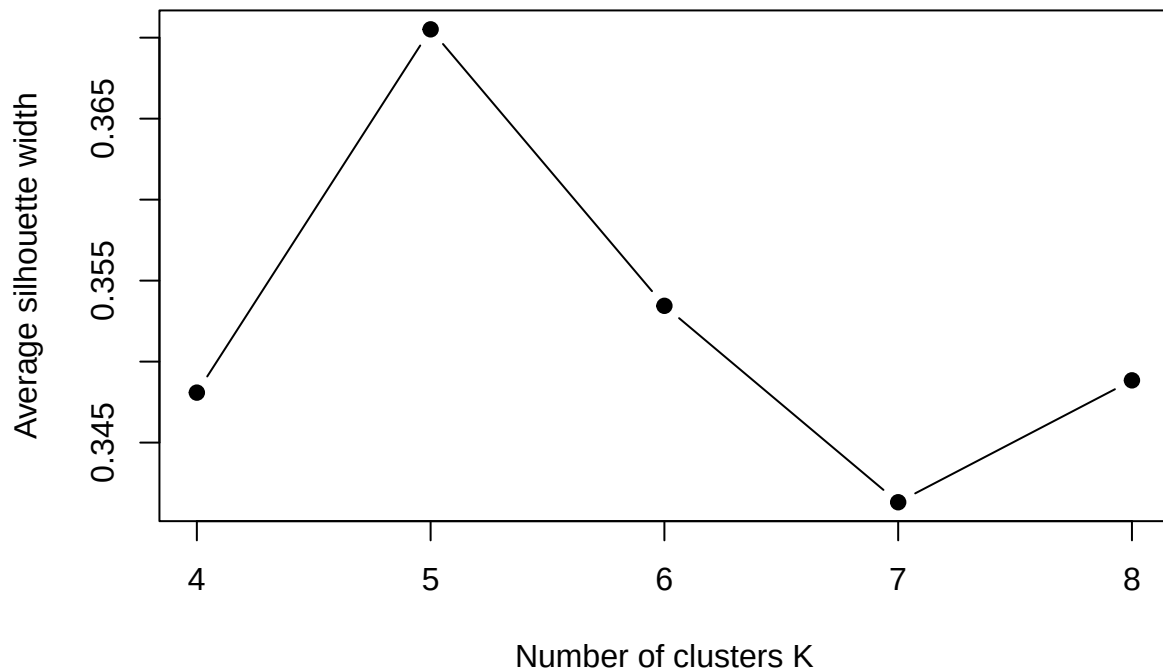
Clustering to Create Classes

```
indices <- which(data$NBA_FG3A >= 10 & data$NBA_FG3_PCT > 0.05) #Use a bit less strict filtering
clust_data <- na.omit(per_min_data[indices, c("NBA_FG3A", "NBA_FG3_PCT")])
clust_data_scaled <- scale(clust_data)

sil_width <- numeric(8)
for (k in 4:8) { #use silhouette method
  set.seed(123)
  km <- kmeans(clust_data_scaled, centers = k, nstart = 25)
  ss <- silhouette(km$cluster, dist(clust_data_scaled))
  sil_width[k] <- mean(ss[, 3])
}

plot(4:8, sil_width[4:8], type = "b", pch = 19, #plot results
     xlab = "Number of clusters K",
     ylab = "Average silhouette width",
     main = "Silhouette Method for Optimal K")
```

Silhouette Method for Optimal K



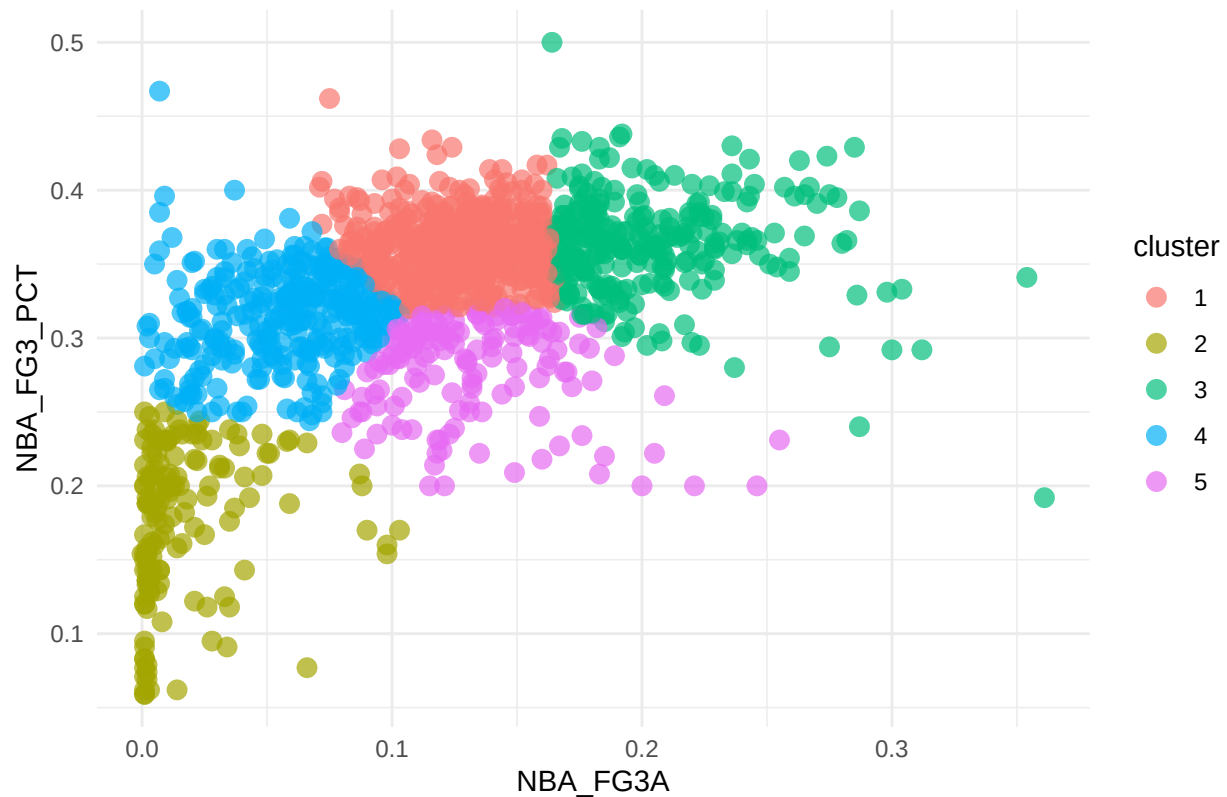
```
best_k <- which.max(sil_width) #set best k

set.seed(123)
km_best <- kmeans(clust_data_scaled, centers = best_k, nstart = 25) #graph the best k clusters

clust_plot <- data.frame(clust_data, cluster = factor(km_best$cluster))

ggplot(clust_plot, aes(x = NBA_FG3A, y = NBA_FG3_PCT, color = cluster)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(title = paste("K-means Clustering with K =", best_k,
                    " (Silhouette Method)")) +
  theme_minimal()
```

K-means Clustering with K = 5 (Silhouette Method)



Individual Clustering

```

y <- per_min_data$NBA_FG3A[indices]
sel <- indices[ which(!is.na(per_min_data$NBA_FG3A[indices])) ]
x <- na.omit(per_min_data$NBA_FG3A[indices])
x_scaled <- scale(x)
d <- dist(x_scaled)
hc <- hclust(d, method = "ward.D2")

# Plot dendrogram with cutoff lines and rectangles for k = 3 to 8
plot(hc, labels = FALSE, hang = -1,
     main = "Hierarchical Clustering Dendrogram (NBA_FG3A)\nwith Cluster Cutoffs (k = 3 to 5)")

# Get the height range of the dendrogram
height_range <- max(hc$height)

# Add horizontal cutoff lines and colored rectangles for each k
colors <- c("red", "blue", "green3")

for (i in 1:length(3:5)) {
  k <- (3:5)[i]

  # Find the height at which cutting produces exactly k clusters
  # This is the midpoint between merge heights for k and k-1 clusters

```

```

if(k < length(hc$height)) {
  cutoff_height <- (hc$height[length(hc$height) - k + 1] +
    hc$height[length(hc$height) - k + 2]) / 2
} else {
  cutoff_height <- hc$height[1] + 0.1 * diff(range(hc$height))
}

# Add horizontal cutoff line
abline(h = cutoff_height, col = colors[i], lty = 2, lwd = 1.5)

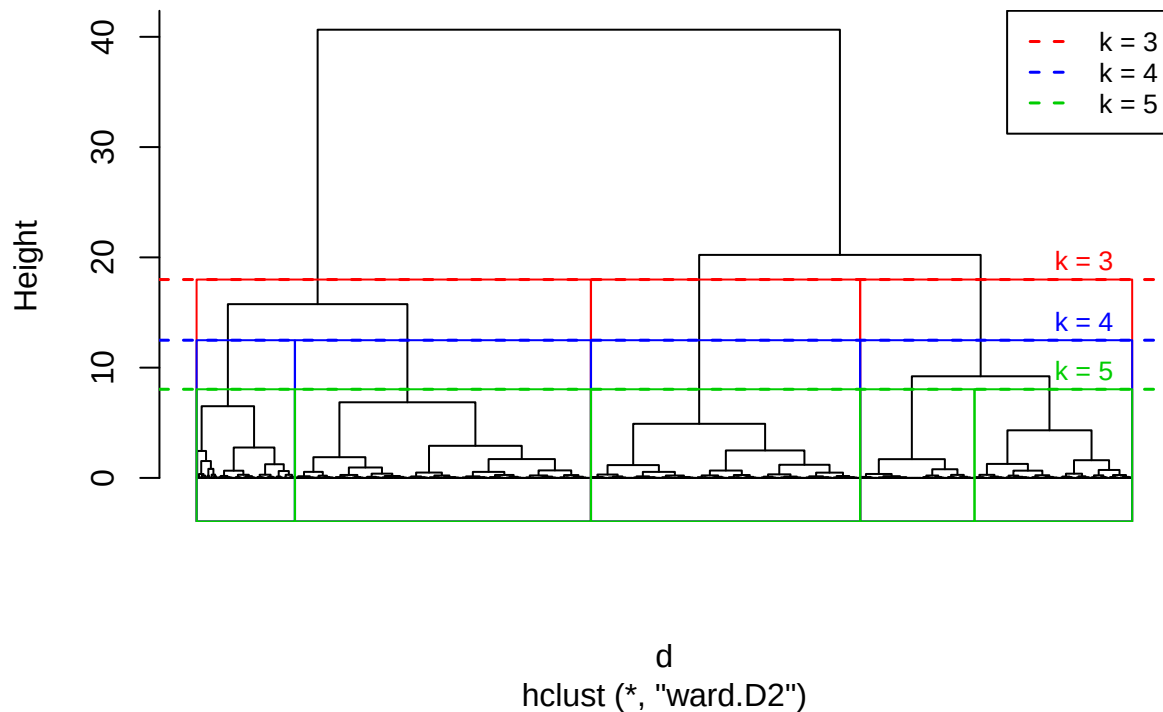
# Add label for the cutoff
text(x = length(hc$order) * 0.98, y = cutoff_height,
  labels = paste0("k = ", k),
  col = colors[i], cex = 0.8, adj = c(1, -0.5))

# Add colored rectangles for the clusters
rect.hclust(hc, k = k, border = colors[i])
}

# Add a legend
legend("topright", legend = paste("k =", 3:5),
  col = colors, lty = 2, lwd = 1.5, cex = 0.8)

```

Hierarchical Clustering Dendrogram (NBA_FG3A) with Cluster Cutoffs (k = 3 to 5)



```

# Compute silhouette widths
sil_width <- numeric(20)

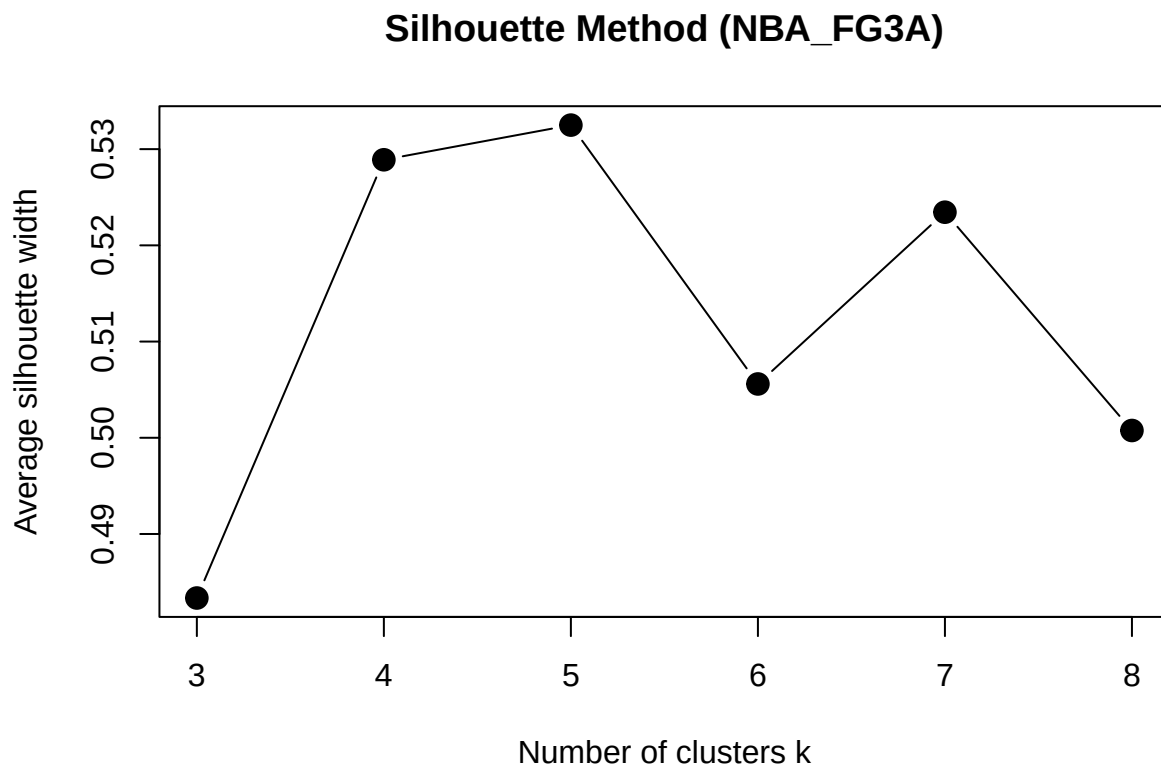
```

```

for (k in 3:8) {
  clusters <- cutree(hc, k)
  ss <- silhouette(clusters, d)
  sil_width[k] <- mean(ss[, 3])
}

# Plot silhouette results
plot(3:8, sil_width[3:8], type = "b", pch = 19, cex = 1.5,
     xlab = "Number of clusters k",
     ylab = "Average silhouette width",
     main = "Silhouette Method (NBA_FG3A)",
     xaxt = "n") # Suppress default x-axis to customize
axis(1, at = 3:8) # Add custom x-axis with integer ticks

```



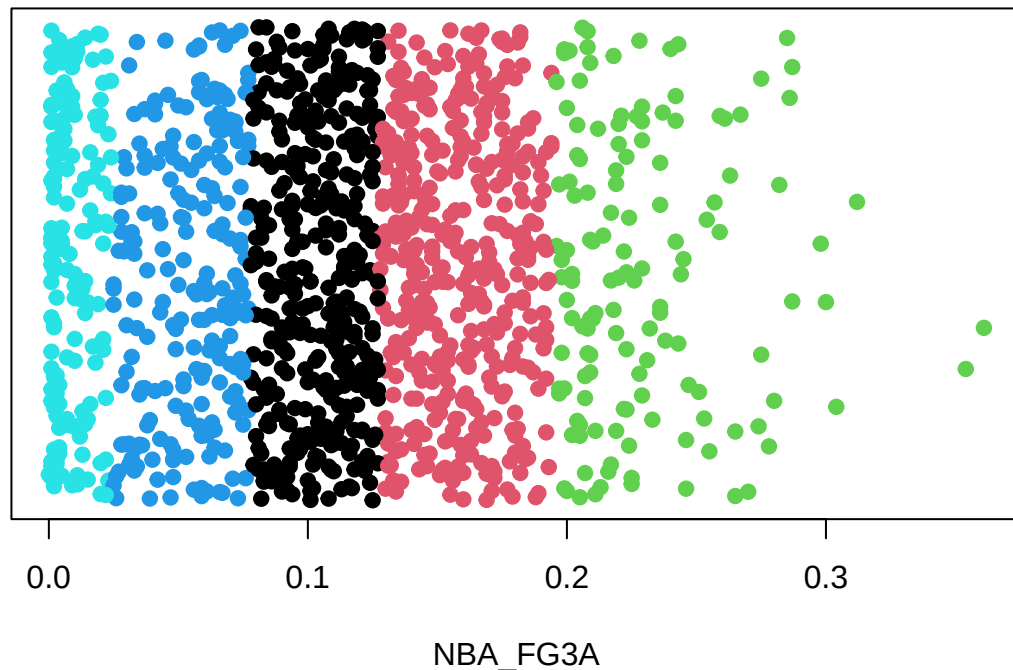
```

best_k <- which.max(sil_width)
clusters_best <- cutree(hc, k = best_k)

plot(
  x,
  jitter(rep(0, length(x)), amount = 0.05),
  col = clusters_best, pch = 19, xlab = "NBA_FG3A", ylab = "",
  yaxt = "n", main = paste("Hierarchical Clustering on NBA_FG3A (k =", best_k, ")")
)

```

Hierarchical Clustering on NBA_FG3A (k = 5)



```
cluster_medians <- tapply(x, clusters_best, median)
ordered_clusters <- names(sort(cluster_medians, decreasing = FALSE))
new_label_map <- setNames(seq_along(ordered_clusters), ordered_clusters)
clusters_ordered <- as.integer(new_label_map[as.character(clusters_best)])
per_min_data$Class_3PA <- NA_integer_
per_min_data$Class_3PA[sel] <- clusters_ordered
per_min_data$Class_3PA[is.na(per_min_data$Class_3PA)] <- 1
per_min_data$Class_3PA <- factor(per_min_data$Class_3PA,
                                levels = as.character(1:best_k),
                                ordered = TRUE)
round(tapply(x, per_min_data$Class_3PA[sel], mean), 3)
```

```
##      1      2      3      4      5
## 0.009 0.054 0.104 0.158 0.230
```

```
table(per_min_data$Class_3PA, useNA = "ifany")
```

```
##
##      1      2      3      4      5
## 344 218 372 409 136
```

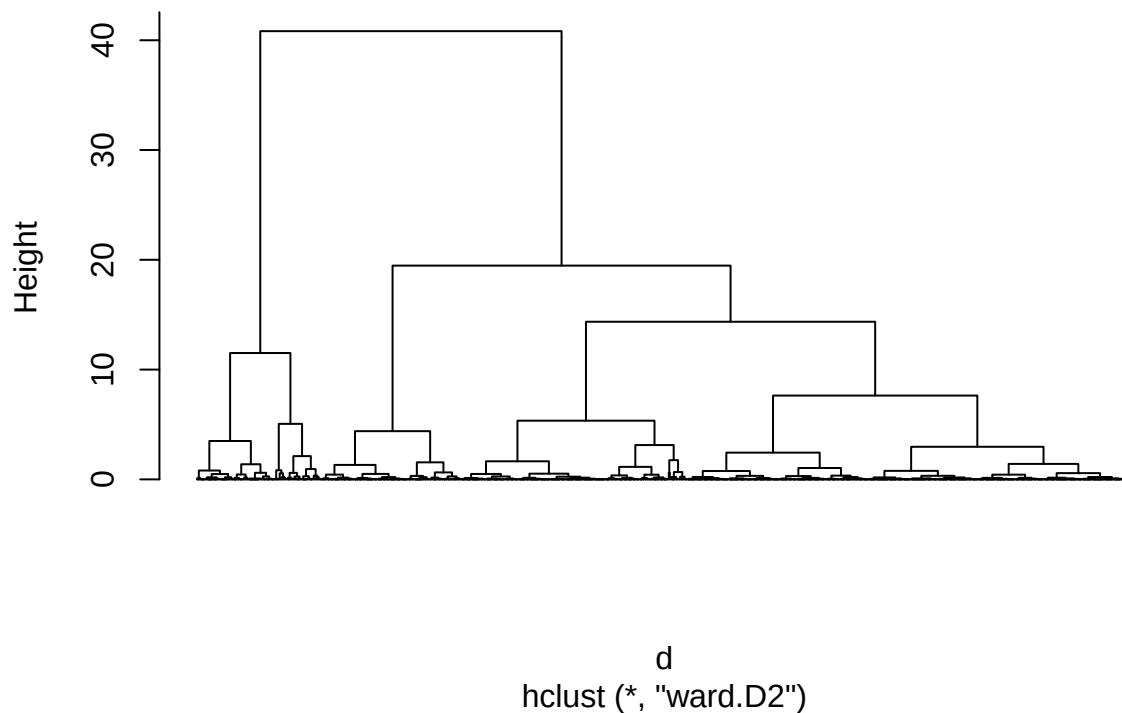
```
#head(per_min_data)
```

```

y <- per_min_data$NBA_FG3_PCT[indices]
not_na_idx <- which(!is.na(y))
sel <- indices[ which(!is.na(per_min_data$NBA_FG3_PCT[indices])) ]
x <- na.omit(data$NBA_FG3_PCT[indices])
x_scaled <- scale(x)
d <- dist(x_scaled)
hc <- hclust(d, method = "ward.D2")
plot(hc, labels = FALSE, hang = -1,
     main = "Hierarchical Clustering Dendrogram (NBA_FG3_PCT)")

```

Hierarchical Clustering Dendrogram (NBA_FG3_PCT)



```

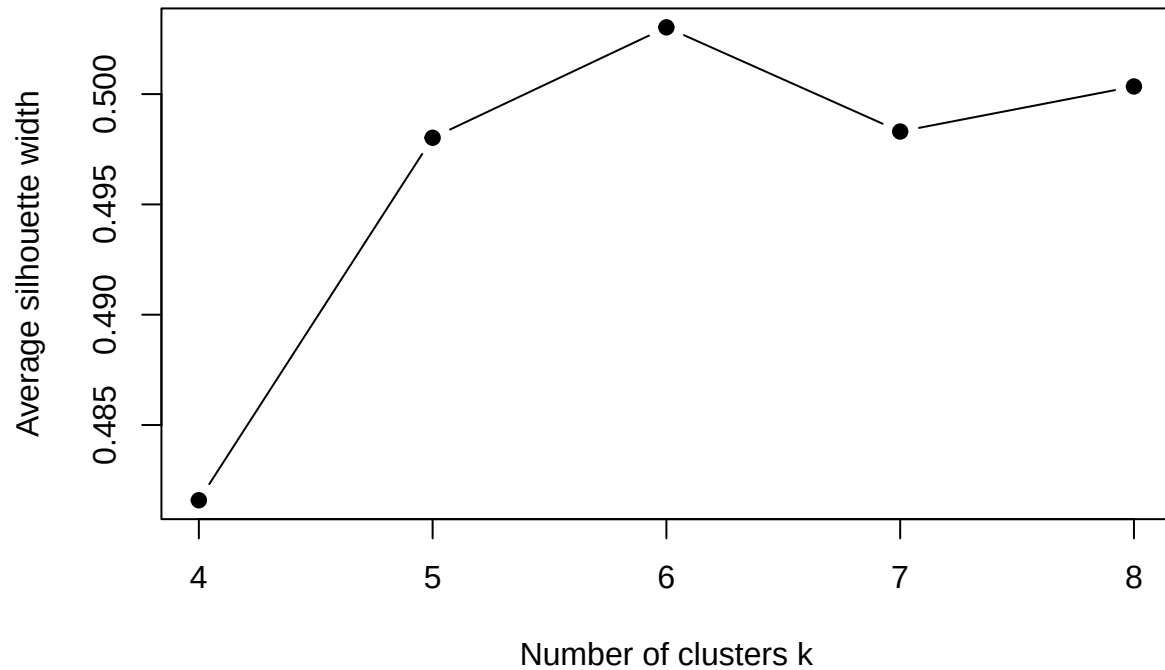
sil_width <- numeric(20)

for (k in 4:8) {
  clusters <- cutree(hc, k)
  ss <- silhouette(clusters, d)
  sil_width[k] <- mean(ss[, 3])
}

plot(4:8, sil_width[4:8], type = "b", pch = 19,
     xlab = "Number of clusters k",
     ylab = "Average silhouette width",
     main = "Silhouette Method (NBA_FG3_PCT)")

```

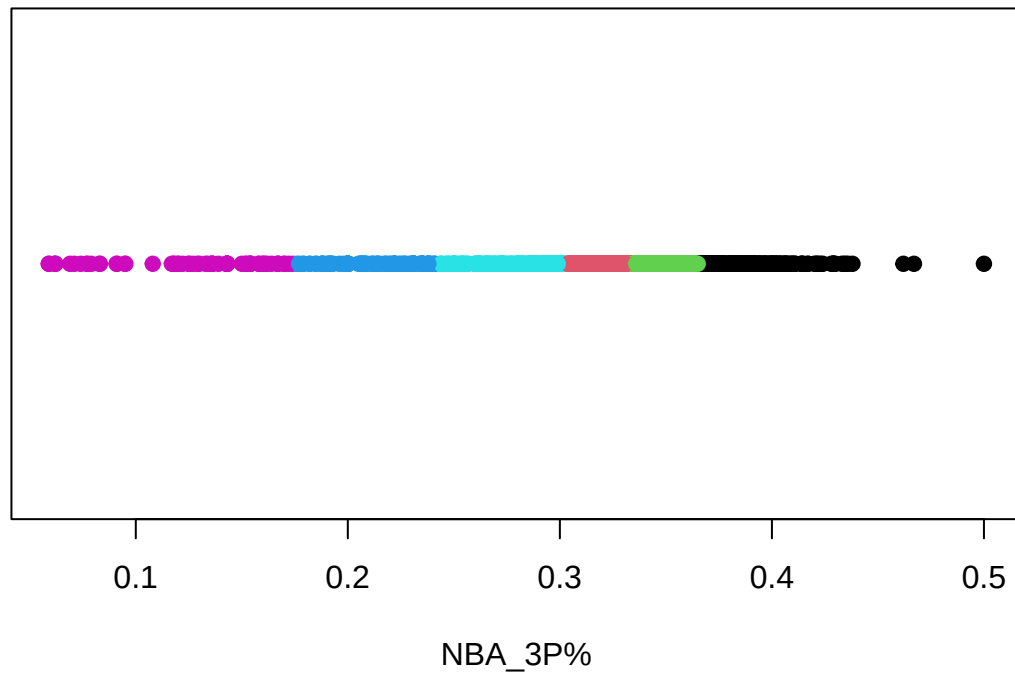
Silhouette Method (NBA_FG3_PCT)



```
best_k <- which.max(sil_width)
clusters_best <- cutree(hc, k = best_k)

plot(x, rep(0, length(x)), col = clusters_best, pch = 19,
     xlab = "NBA_3P%", ylab = "",
     yaxt = "n", main = paste("Hierarchical Clustering on NBA_FG3_PCT (k =", best_k, ")"))
```

Hierarchical Clustering on NBA_FG3_PCT (k = 6)



```
cluster_medians <- tapply(x, clusters_best, median)
ordered_clusters <- names(sort(cluster_medians, decreasing = FALSE))
new_label_map <- setNames(seq_along(ordered_clusters), ordered_clusters)
clusters_ordered <- as.integer(new_label_map[as.character(clusters_best)])
per_min_data$Class_3P_PCT <- NA_integer_
per_min_data$Class_3P_PCT[sel] <- clusters_ordered
per_min_data$Class_3P_PCT[is.na(per_min_data$Class_3P_PCT)] <- 1
per_min_data$Class_3P_PCT <- factor(per_min_data$Class_3P_PCT,
                                   levels = as.character(1:best_k),
                                   ordered = TRUE)
round(tapply(x, per_min_data$Class_3P_PCT[sel], median), 3)
```

```
##      1      2      3      4      5      6
## 0.136 0.214 0.286 0.321 0.350 0.382
```

```
table(per_min_data$Class_3P_PCT, useNA = "ifany")
```

```
##
##      1      2      3      4      5      6
## 250 108 194 251 364 312
```

```
#head(per_min_data)
```

Assign Clusters to the dataset

```
# Assign the cluster numbers
per_min_data$Cluster3Shooting <- NA_integer_
valid_rows <- indices[complete.cases(per_min_data[indices, c("NBA_FG3A", "NBA_FG3_PCT")])]
per_min_data$Cluster3Shooting[valid_rows] <- km_best$cluster
table(per_min_data$Cluster3Shooting)
```

```
##
##   1   2   3   4   5
## 425 148 267 304 149
```

```
# Rearrange for any functions that may still use ordinal
per_min_data$Cluster3Shooting_reordered <- dplyr::case_when(
  per_min_data$Cluster3Shooting == 2 ~ 1L,
  per_min_data$Cluster3Shooting == 5 ~ 2L,
  per_min_data$Cluster3Shooting == 4 ~ 3L,
  per_min_data$Cluster3Shooting == 1 ~ 4L,
  per_min_data$Cluster3Shooting == 3 ~ 5L,
  TRUE ~ 1L
)

per_min_data$Cluster3Shooting_reordered <- dplyr::case_when(
  per_min_data$NBA_FG3_PCT < 0.3 & per_min_data$Cluster3Shooting_reordered == 5L ~ 2L,
  TRUE ~ per_min_data$Cluster3Shooting_reordered
)
```

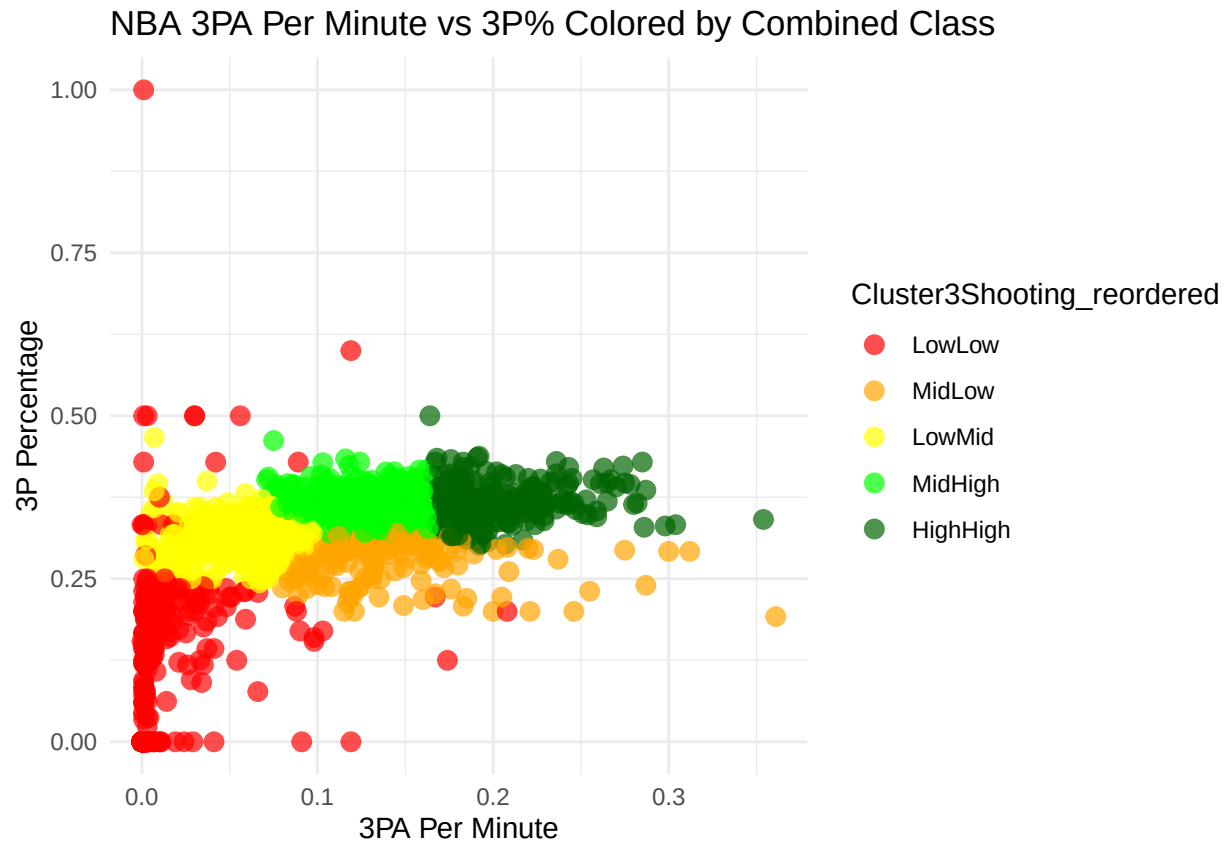
```
# Check summary
table(per_min_data$Cluster3Shooting_reordered)
```

```
##
##   1   2   3   4   5
## 334 159 304 425 257
```

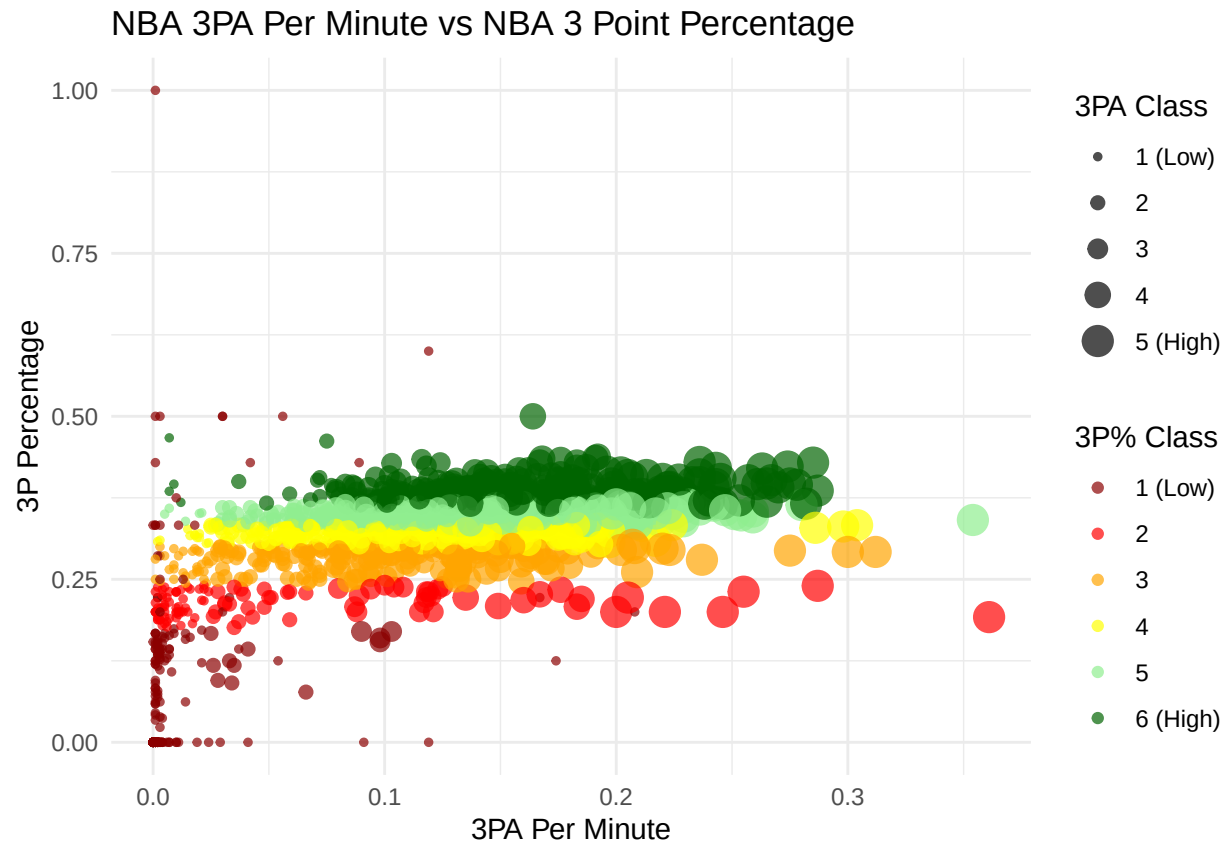
```
# Set labels
class_levels <- c("LowLow", "MidLow", "LowMid", "MidHigh", "HighHigh")
per_min_data$Cluster3Shooting_reordered <- factor(per_min_data$Cluster3Shooting_reordered,
  levels = 1:5,
  labels = class_levels,
  ordered = TRUE)
```

```
# Graph clusters with better colors
ggplot(per_min_data, aes(x = NBA_FG3A, y = NBA_FG3_PCT, color = Cluster3Shooting_reordered)) +
  geom_point(size = 3, alpha = 0.7) +
  scale_color_manual(
    values = c("LowLow" = "red",
              "MidLow" = "orange",
              "LowMid" = "yellow",
              "MidHigh" = "green",
              "HighHigh" = "darkgreen")
  ) +
```

```
labs(
  title = "NBA 3PA Per Minute vs 3P% Colored by Combined Class",
  x = "3PA Per Minute",
  y = "3P Percentage"
) +
theme_minimal()
```



```
ggplot(per_min_data, aes(x = NBA_FG3A, y = NBA_FG3_PCT)) +
  geom_point(aes(color = factor(Class_3P_PCT), size = factor(Class_3PA)), alpha = 0.7) +
  scale_color_manual(name = "3P% Class", values = c("1" = "darkred", "2" = "red", "3" = "orange",
    "4" = "yellow", "5" = "lightgreen", "6" = "darkgreen"),
    labels = c("1 (Low)", "2", "3", "4", "5", "6 (High)")) +
  scale_size_manual(name = "3PA Class", values = c("1" = 1, "2" = 2, "3" = 3, "4" = 4,
    "5" = 5),
    labels = c("1 (Low)", "2", "3", "4", "5 (High)")) +
  labs(title = "NBA 3PA Per Minute vs NBA 3 Point Percentage", x = "3PA Per Minute", y = "3P Percentage")
theme_minimal()
```



Prediction on both 3PA and 3P% Class

Initialize Train / Test Split

```
# Specific set of predictors mentioned in report
X <- per_min_data[, c(8,13,38:40,42,43,45,46,48:50, 52:55, 56:66)]
X[is.na(X)] <- 0
y <- factor(per_min_data[, ncol(per_min_data)], levels = class_levels)

df <- data.frame(X, response = y)
df_scaled <- data.frame(scale(df[, -ncol(df)]), response = df$response)

# Split train and test
set.seed(67)
train_index <- createDataPartition(df_scaled$response, p = 0.8, list = FALSE)
train_data <- df_scaled[train_index, ]
test_data <- df_scaled[-train_index, ]
```

Backwards Stepwise Selection

Check for collinearity

```
vif(step_model)
```

```
##      y>=MidLow  y>=LowMid  y>=MidHigh y>=HighHigh    PRE_3PA  PRE_3P_PCT
##      1.978648   2.304373   1.549787   1.331524   1.769551   1.339271
## PRE_FT_PCT    PRE_TOV
##      1.523065   1.026334
```

```
predictors <- df_scaled[, c("PRE_FGA", "PRE_FG_PCT", "PRE_3PA",
                             "PRE_FTA", "PRE_DRB", "PRE_BLK", "PRE_TOV", "PRE_PTS")]
cor_matrix <- cor(predictors)
print(round(cor_matrix, 3))
```

```
##      PRE_FGA PRE_FG_PCT PRE_3PA PRE_FTA PRE_DRB PRE_BLK PRE_TOV PRE_PTS
## PRE_FGA      1.000   -0.230   0.434   0.337  -0.039  -0.190   0.279   0.853
## PRE_FG_PCT  -0.230    1.000  -0.699   0.323   0.368   0.609  -0.035   0.151
## PRE_3PA      0.434   -0.699   1.000  -0.350  -0.304  -0.579  -0.083   0.206
## PRE_FTA      0.337    0.323  -0.350   1.000   0.185   0.245   0.408   0.615
## PRE_DRB     -0.039    0.368  -0.304   0.185   1.000   0.373   0.005   0.079
## PRE_BLK     -0.190    0.609  -0.579   0.245   0.373   1.000  -0.016   0.002
## PRE_TOV      0.279   -0.035  -0.083   0.408   0.005  -0.016   1.000   0.278
## PRE_PTS      0.853    0.151   0.206   0.615   0.079   0.002   0.278   1.000
```

Our main issue is between PPG and FGA, check both.

```
noPTS <- polr(response ~ PRE_FGA+PRE_FG_PCT+PRE_3PA+PRE_FTA+PRE_DRB+PRE_BLK+PRE_TOV, data = train_data,
noFGA <- polr(response ~ PRE_FG_PCT+PRE_3PA+PRE_FTA+PRE_DRB+PRE_BLK+PRE_TOV+PRE_PTS, data = train_data,
AIC(noPTS);AIC(noFGA)
```

```
## [1] 3092.504
```

```
## [1] 3084.665
```

Because the no FGA model has the lower AIC, it is the better model because we chose AIC as our criterion.

```
summary(noFGA)
```

```
## Call:
## polr(formula = response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
##      PRE_BLK + PRE_TOV + PRE_PTS, data = train_data, Hess = TRUE)
##
## Coefficients:
##      Value Std. Error t value
## PRE_FG_PCT -0.1289    0.10210  -1.262
## PRE_3PA      1.0900    0.12490   8.727
```

```
## PRE_FTA      -0.3250      0.09995  -3.252
## PRE_DRB       0.1751      0.06435   2.721
## PRE_BLK      -0.4367      0.08509  -5.132
## PRE_TOV      -0.2594      0.06238  -4.158
## PRE_PTS       0.2969      0.10379   2.860
##
## Intercepts:
##              Value      Std. Error t value
## LowLow|MidLow   -1.7440     0.0876  -19.9160
## MidLow|LowMid   -0.9783     0.0754  -12.9824
## LowMid|MidHigh    0.1876     0.0693   2.7093
## MidHigh|HighHigh  2.1728     0.0937  23.1858
##
## Residual Deviance: 3062.665
## AIC: 3084.665
```

```
vif(noFGA)
```

```
##          PRE_FG_PCT          PRE_3PA          PRE_FTA          PRE_DRB
##          3.090255          4.184378          3.083794          1.320215
##          PRE_BLK          PRE_TOV          PRE_PTS      LowLow|MidLow
##          1.808852          1.302522          3.477962          2.097783
##      MidLow|LowMid      LowMid|MidHigh      MidHigh|HighHigh
##          2.458664          1.606307          1.334068
```

```
predictors <- df_scaled[, c("PRE_FG_PCT", "PRE_3PA",
  "PRE_FTA", "PRE_DRB", "PRE_BLK", "PRE_TOV", "PRE_PTS")]
cor_matrix <- cor(predictors)
print(round(cor_matrix, 3))
```

```
##          PRE_FG_PCT PRE_3PA PRE_FTA PRE_DRB PRE_BLK PRE_TOV PRE_PTS
## PRE_FG_PCT      1.000 -0.699  0.323  0.368  0.609 -0.035  0.151
## PRE_3PA         -0.699  1.000 -0.350 -0.304 -0.579 -0.083  0.206
## PRE_FTA         0.323 -0.350  1.000  0.185  0.245  0.408  0.615
## PRE_DRB         0.368 -0.304  0.185  1.000  0.373  0.005  0.079
## PRE_BLK         0.609 -0.579  0.245  0.373  1.000 -0.016  0.002
## PRE_TOV        -0.035 -0.083  0.408  0.005 -0.016  1.000  0.278
## PRE_PTS         0.151  0.206  0.615  0.079  0.002  0.278  1.000
```

Initializing Storing Results

```
model_results <- data.frame(
  Model = character(),
  Accuracy = numeric(),
  Precision = numeric(),
  Recall = numeric(),
  F1 = numeric(),
  AUC = numeric(),
  stringsAsFactors = FALSE
)
```

LDA

```
set.seed(22222)
# Run cross validation
train_control <- trainControl(method = "cv", number = 10,
                              summaryFunction = multiClassSummary)

lda_cv <- train(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
                PRE_BLK + PRE_TOV + PRE_PTS, data = train_data,
                method = "lda",
                trControl = train_control)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

print(lda_cv)

## Linear Discriminant Analysis
##
## 1186 samples
## 7 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1069, 1068, 1067, 1067, 1067, 1066, ...
## Resampling results:
##
##   Accuracy   Kappa     Mean_F1  Mean_Sensitivity  Mean_Specificity
## 0.5002687 0.3445918  NaN      0.4398053      0.8680822
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## NaN              0.87167              NaN          0.4398053
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.1000537           0.6539438

cat("Cross-validated Accuracy (training set):", max(lda_cv$results$Accuracy), "\n")

## Cross-validated Accuracy (training set): 0.5002687

# Set fit model
lda_fit <- lda(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
               PRE_BLK + PRE_TOV + PRE_PTS,
               data = train_data)

pred_obj <- predict(lda_fit, test_data)

all_pred <- factor(pred_obj$class, levels = class_levels)
all_actual <- factor(test_data$response, levels = class_levels)
all_probs <- pred_obj$posterior

# Find CM
```

```
conf_mat_lda <- confusionMatrix(all_pred, all_actual, mode = "prec_recall")
print(conf_mat_lda)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      44      2      13      10      1
##   MidLow       0      0       0       0      0
##   LowMid      13      1      21      12      3
##   MidHigh      9     22      26      41     18
##   HighHigh     0      6       0      22     29
##
## Overall Statistics
##
##           Accuracy : 0.4608
##           95% CI : (0.4026, 0.5197)
##   No Information Rate : 0.2901
##   P-Value [Acc > NIR] : 4.867e-10
##
##           Kappa : 0.2928
##
##   McNemar's Test P-Value : 1.322e-05
##
## Statistics by Class:
##
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision           0.6286           NA           0.42000           0.3534
## Recall              0.6667           0.0000           0.35000           0.4824
## F1                  0.6471           NA           0.38182           0.4080
## Prevalence          0.2253           0.1058           0.20478           0.2901
## Detection Rate      0.1502           0.0000           0.07167           0.1399
## Detection Prevalence 0.2389           0.0000           0.17065           0.3959
## Balanced Accuracy    0.7761           0.5000           0.61277           0.5609
##
##           Class: HighHigh
## Precision           0.50877
## Recall              0.56863
## F1                  0.53704
## Prevalence          0.17406
## Detection Rate      0.09898
## Detection Prevalence 0.19454
## Balanced Accuracy    0.72646
```

```
# Calculate Accuracy and AUC
```

```
LDA_acc <- mean(as.character(all_pred) == as.character(all_actual))
cat("Test Accuracy (LDA):", LDA_acc, "\n")
```

```
## Test Accuracy (LDA): 0.4607509
```

```
overall_multiclass <- multiclass.roc(all_actual, all_probs)
cat("Overall multiclass AUC (test set):", overall_multiclass$auc, "\n")
```

```
## Overall multiclass AUC (test set): 0.7513292
```

```

# Graph AUC / Calculate Macro AUC
fpr_grid <- seq(0, 1, length.out = 200)
tpr_matrix <- matrix(NA, nrow = length(fpr_grid), ncol = length(class_levels))

for (i in seq_along(class_levels)) {
  class <- class_levels[i]
  binary_actual <- factor(ifelse(all_actual == class, class, "Other"),
                           levels = c("Other", class))
  class_probs <- all_probs[, class]

  roc_obj <- roc(response = binary_actual,
                 predictor = class_probs,
                 levels = c("Other", class),
                 positive = class,
                 quiet = TRUE)

  cat("AUC of", class, ":", auc(roc_obj), "\n")

  fpr <- 1 - roc_obj$specificities
  tpr <- roc_obj$sensitivities

  interp_tpr <- approx(fpr, tpr, xout = fpr_grid, ties = mean, rule = 2)$y
  tpr_matrix[, i] <- interp_tpr
}

```

```

## AUC of LowLow : 0.8869977
## AUC of MidLow : 0.7089387
## AUC of LowMid : 0.7186695
## AUC of MidHigh : 0.6300905
## AUC of HighHigh : 0.8317939

```

```

macro_tpr_lda <- rowMeans(tpr_matrix, na.rm = TRUE)
macro_auc_lda <- trapz(fpr_grid, macro_tpr_lda)
cat("Macro AUC (LDA test set):", macro_auc_lda, "\n")

```

```

## Macro AUC (LDA test set): 0.7552616

```

```

model_results <- rbind(model_results, data.frame(
  Model = "LDA",
  Accuracy = LDA_acc,
  Precision = mean(conf_mat_lda$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_lda$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_lda$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc_lda
))

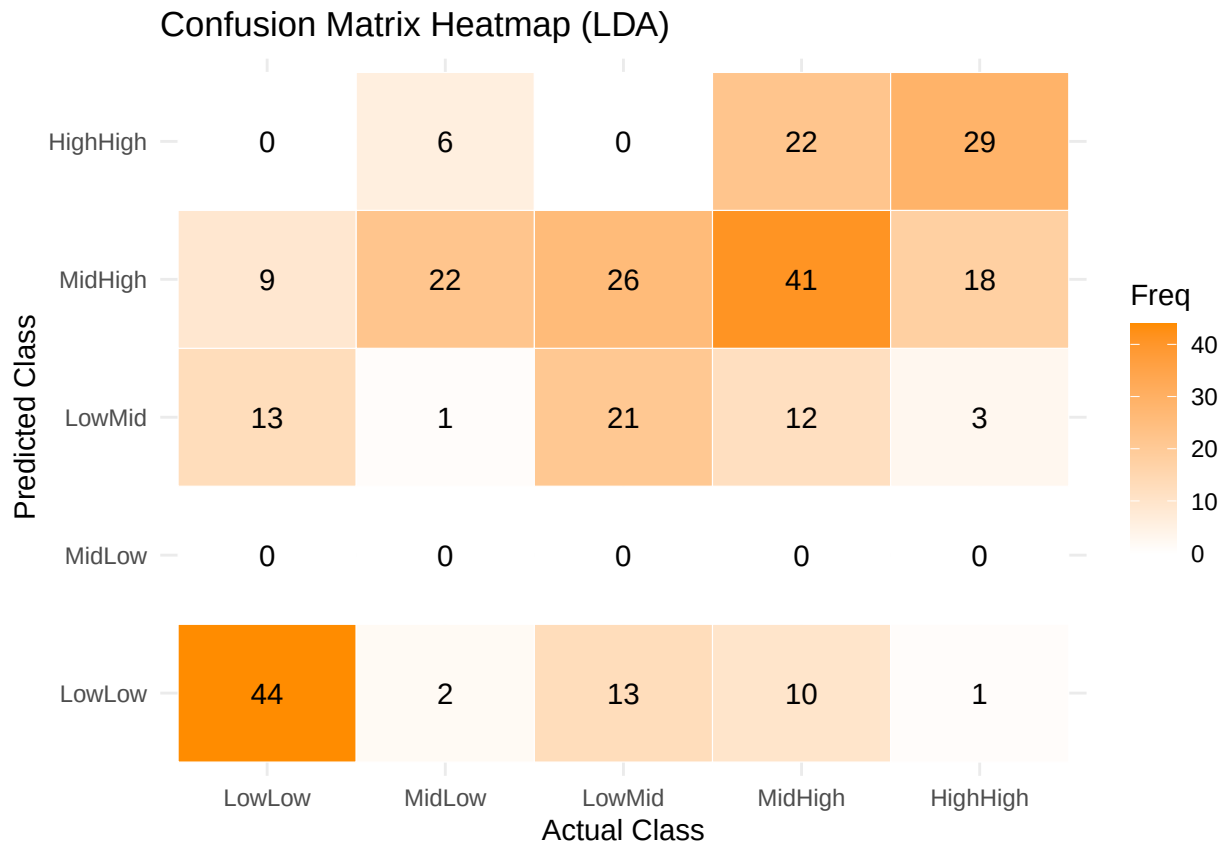
```

```

cm_table <- as.data.frame(conf_mat_lda$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +

```

```
labs(title = "Confusion Matrix Heatmap (LDA)",
     x = "Actual Class", y = "Predicted Class")
```



```
# LDA Decision Boundary Visualization
# Project data onto first two LDA dimensions
lda_proj_train <- as.data.frame(predict(lda_fit, train_data)$x[, 1:2])
lda_proj_train$class <- train_data$response
colnames(lda_proj_train)[1:2] <- c("LD1", "LD2")

# Create grid for decision regions
ld1_range <- seq(min(lda_proj_train$LD1) - 1, max(lda_proj_train$LD1) + 1, length.out = 150)
ld2_range <- seq(min(lda_proj_train$LD2) - 1, max(lda_proj_train$LD2) + 1, length.out = 150)
grid <- expand.grid(LD1 = ld1_range, LD2 = ld2_range)

# Get training means for each predictor
train_means <- colMeans(train_data[, c("PRE_FG_PCT", "PRE_3PA", "PRE_FTA", "PRE_DRB",
                                         "PRE_BLK", "PRE_TOV", "PRE_PTS")])

# Reconstruct original variables from LD scores
grid_orig <- matrix(NA, nrow = nrow(grid), ncol = 7)
colnames(grid_orig) <- names(train_means)

for(i in 1:nrow(grid)) {
  grid_orig[i, ] <- train_means + (grid$LD1[i] * lda_fit$scaling[, 1] +
                                    grid$LD2[i] * lda_fit$scaling[, 2])
}
```

```

}

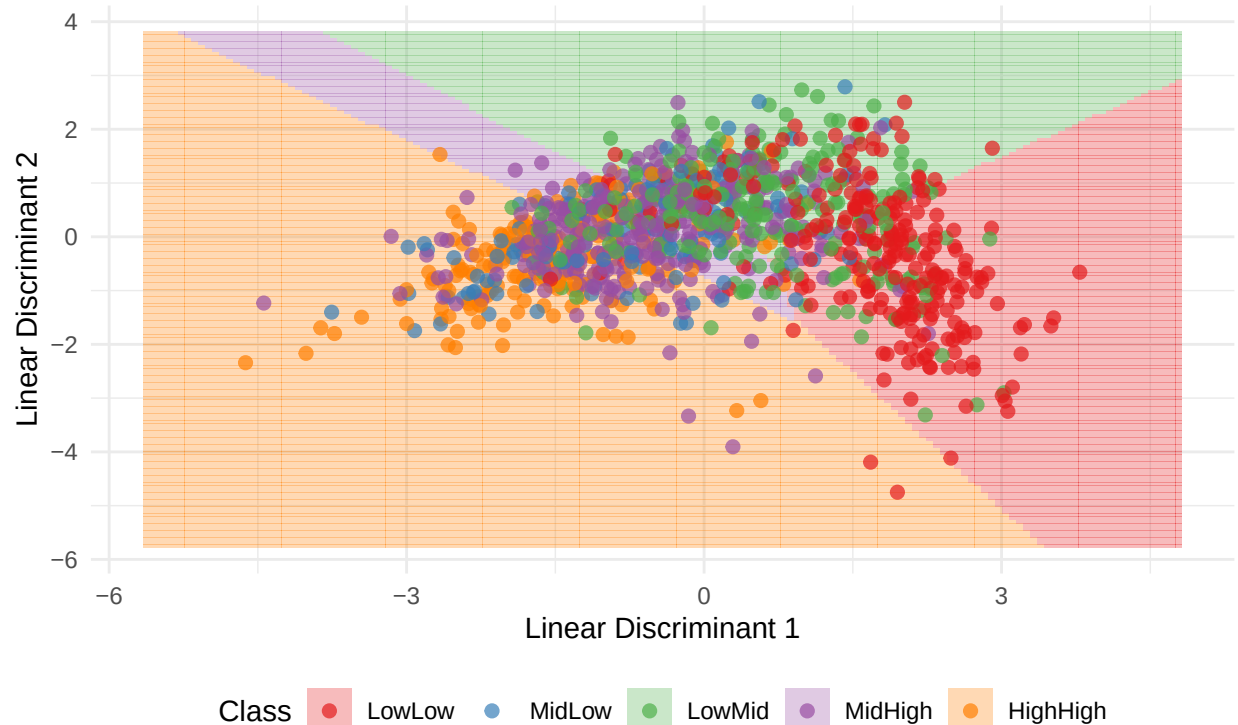
grid_pred <- predict(lda_fit, as.data.frame(grid_orig))$class
grid$class <- factor(grid_pred, levels = class_levels)

# Plot
ggplot() +
  geom_tile(data = grid, aes(x = LD1, y = LD2, fill = class), alpha = 0.3) +
  geom_point(data = lda_proj_train, aes(x = LD1, y = LD2, color = class), size = 2, alpha = 0.7) +
  scale_fill_manual(values = c("#E41A1C", "#377EB8", "#4DAF4A", "#984EA3", "#FF7F00"),
                    drop = FALSE) +
  scale_color_manual(values = c("#E41A1C", "#377EB8", "#4DAF4A", "#984EA3", "#FF7F00"),
                    drop = FALSE) +
  labs(title = "LDA Decision Regions for Combined Class",
       subtitle = paste0("Cross-validated Accuracy: ", round(max(lda_cv$results$Accuracy), 3)),
       x = "Linear Discriminant 1",
       y = "Linear Discriminant 2",
       fill = "Class",
       color = "Class") +
  theme_minimal() +
  theme(legend.position = "bottom",
        plot.title = element_text(face = "bold", size = 14),
        plot.subtitle = element_text(size = 11))

```

LDA Decision Regions for Combined Class

Cross-validated Accuracy: 0.5



QDA

```
# Same steps as LDA
set.seed(22222)
train_control <- trainControl(method = "cv", number = 10,
                              summaryFunction = multiClassSummary)

qda_cv <- train(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
                PRE_BLK + PRE_TOV + PRE_PTS,
                data = train_data,
                method = "qda",
                trControl = train_control)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
print(qda_cv)
```

```
## Quadratic Discriminant Analysis
##
## 1186 samples
## 7 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1069, 1068, 1067, 1067, 1067, 1066, ...
## Resampling results:
##
## Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 0.4486229 0.278488 NaN 0.3958338 0.8548818
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.3557292 0.8584005 0.3557292 0.3958338
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.08972457 0.6253578
```

```
cat("Cross-validated Accuracy (training set):", max(qda_cv$results$Accuracy), "\n")
```

```
## Cross-validated Accuracy (training set): 0.4486229
```

```
qda_fit <- qda(response ~ PRE_FG_PCT + PRE_3PA + PRE_FTA + PRE_DRB +
                PRE_BLK + PRE_TOV + PRE_PTS,
                data = train_data)
```

```
pred_obj <- predict(qda_fit, test_data)
```

```
all_pred <- factor(pred_obj$class, levels = class_levels)
all_actual <- factor(test_data$response, levels = class_levels)
all_probs <- pred_obj$posterior
```

```
conf_mat_qda <- confusionMatrix(all_pred, all_actual, mode = "prec_recall")
print(conf_mat_qda)
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction LowLow MidLow LowMid MidHigh HighHigh
```

```
##   LowLow      44      4      13      11      1
```

```
##   MidLow       0      1       1       2       0
```

```
##   LowMid      10      1      12       4       4
```

```
##   MidHigh     11     18     32     52     16
```

```
##   HighHigh     1      7       2     16     30
```

```
##
```

```
## Overall Statistics
```

```
##
```

```
##           Accuracy : 0.4744
```

```
##           95% CI : (0.416, 0.5333)
```

```
##   No Information Rate : 0.2901
```

```
##   P-Value [Acc > NIR] : 2.178e-11
```

```
##
```

```
##           Kappa : 0.3082
```

```
##
```

```
##   McNemar's Test P-Value : 1.100e-06
```

```
##
```

```
## Statistics by Class:
```

```
##
```

```
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
```

```
## Precision           0.6027      0.250000      0.38710      0.4031
```

```
## Recall              0.6667      0.032258      0.20000      0.6118
```

```
## F1                  0.6331      0.057143      0.26374      0.4860
```

```
## Prevalence          0.2253      0.105802      0.20478      0.2901
```

```
## Detection Rate      0.1502      0.003413      0.04096      0.1775
```

```
## Detection Prevalence 0.2491      0.013652      0.10580      0.4403
```

```
## Balanced Accuracy    0.7695      0.510404      0.55923      0.6208
```

```
##           Class: HighHigh
```

```
## Precision           0.5357
```

```
## Recall              0.5882
```

```
## F1                  0.5607
```

```
## Prevalence          0.1741
```

```
## Detection Rate      0.1024
```

```
## Detection Prevalence 0.1911
```

```
## Balanced Accuracy    0.7404
```

```
QDA_acc <- mean(as.character(all_pred) == as.character(all_actual))
```

```
cat("Test Accuracy (QDA):", QDA_acc, "\n")
```

```
## Test Accuracy (QDA): 0.4744027
```

```
overall_multiclass <- multiclass.roc(all_actual, all_probs)
```

```
cat("Overall multiclass AUC (QDA test set):", overall_multiclass$auc, "\n")
```

```
## Overall multiclass AUC (QDA test set): 0.7344629
```

```
fpr_grid <- seq(0, 1, length.out = 200)
tpr_matrix <- matrix(NA, nrow = length(fpr_grid), ncol = length(class_levels))

for (i in seq_along(class_levels)) {
  class <- class_levels[i]
  binary_actual <- factor(ifelse(all_actual == class, class, "Other"),
                           levels = c("Other", class))
  class_probs <- all_probs[, class]

  roc_obj <- roc(response = binary_actual,
                 predictor = class_probs,
                 levels = c("Other", class),
                 positive = class,
                 quiet = TRUE)

  cat("AUC of", class, ":", auc(roc_obj), "\n")

  fpr <- 1 - roc_obj$specificities
  tpr <- roc_obj$sensitivities

  interp_tpr <- approx(fpr, tpr, xout = fpr_grid, ties = mean, rule = 2)$y
  tpr_matrix[, i] <- interp_tpr
}
```

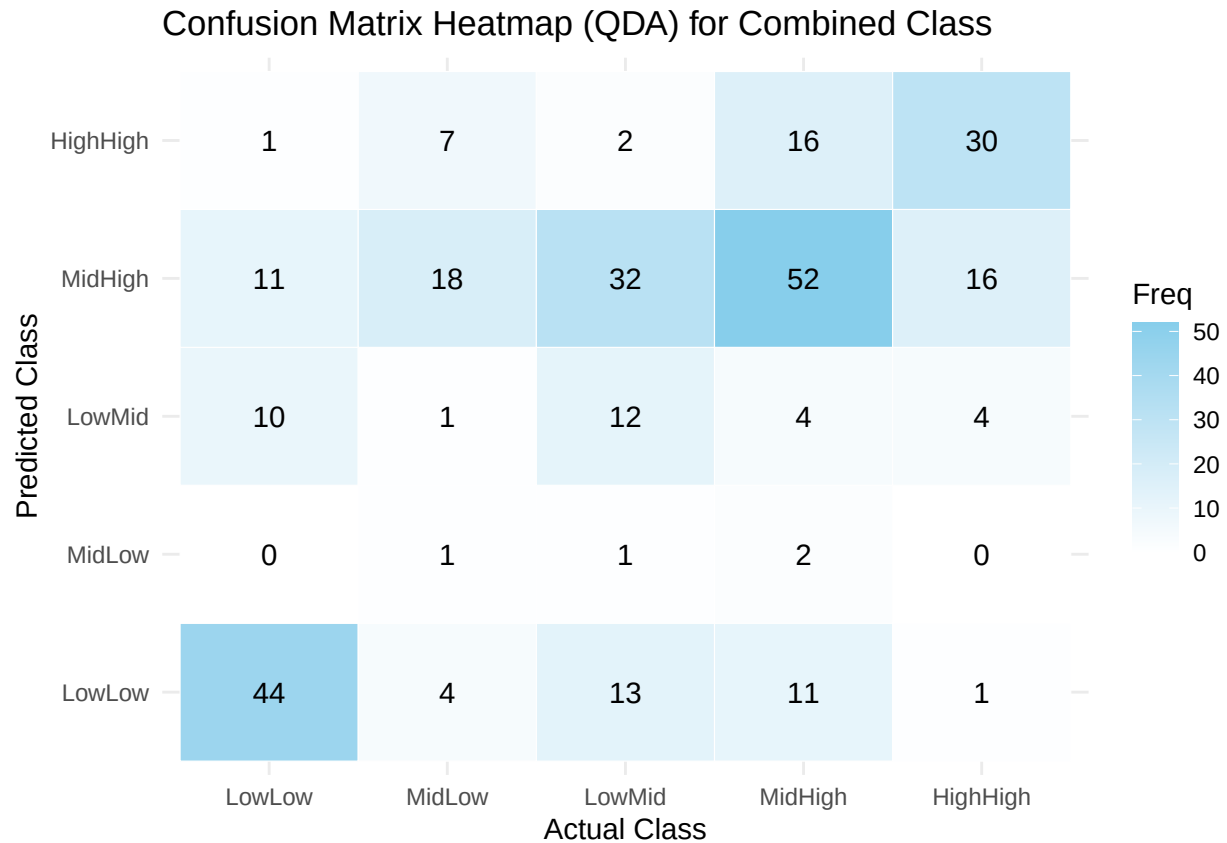
```
## AUC of LowLow : 0.8817247
## AUC of MidLow : 0.6381433
## AUC of LowMid : 0.6808298
## AUC of MidHigh : 0.6595023
## AUC of HighHigh : 0.8197213
```

```
macro_tpr_qda <- rowMeans(tpr_matrix, na.rm = TRUE)
macro_auc_qda <- trapz(fpr_grid, macro_tpr_qda)
cat("Macro AUC (QDA test set):", macro_auc_qda, "\n")
```

```
## Macro AUC (QDA test set): 0.7359076
```

```
model_results <- rbind(model_results, data.frame(
  Model = "QDA",
  Accuracy = QDA_acc,
  Precision = mean(conf_mat_qda$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_qda$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_qda$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc_qda
))
```

```
cm_table <- as.data.frame(conf_mat_qda$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (QDA) for Combined Class",
       x = "Actual Class", y = "Predicted Class")
```



```
# QDA Decision Boundary Visualization
# Project data using LDA dimensions for visualization purposes
qda_proj_train <- as.data.frame(as.matrix(train_data[, c("PRE_FG_PCT", "PRE_3PA", "PRE_FTA", "PRE_DRB",
                                                         "PRE_BLK", "PRE_TOV", "PRE_PTS")])) %*%
  lda_fit$scaling[, 1:2])
qda_proj_train$class <- train_data$response
colnames(qda_proj_train)[1:2] <- c("LD1", "LD2")

# Create grid for decision regions
ld1_range <- seq(min(qda_proj_train$LD1) - 1, max(qda_proj_train$LD1) + 1, length.out = 150)
ld2_range <- seq(min(qda_proj_train$LD2) - 1, max(qda_proj_train$LD2) + 1, length.out = 150)
grid <- expand.grid(LD1 = ld1_range, LD2 = ld2_range)

# Get training means for each predictor
train_means <- colMeans(train_data[, c("PRE_FG_PCT", "PRE_3PA", "PRE_FTA", "PRE_DRB",
                                         "PRE_BLK", "PRE_TOV", "PRE_PTS")])

# Reconstruct original variables from LD scores
grid_orig <- matrix(NA, nrow = nrow(grid), ncol = 7)
colnames(grid_orig) <- names(train_means)

for(i in 1:nrow(grid)) {
  grid_orig[i, ] <- train_means + (grid$LD1[i] * lda_fit$scaling[, 1] +
                                   grid$LD2[i] * lda_fit$scaling[, 2])
}
```

```

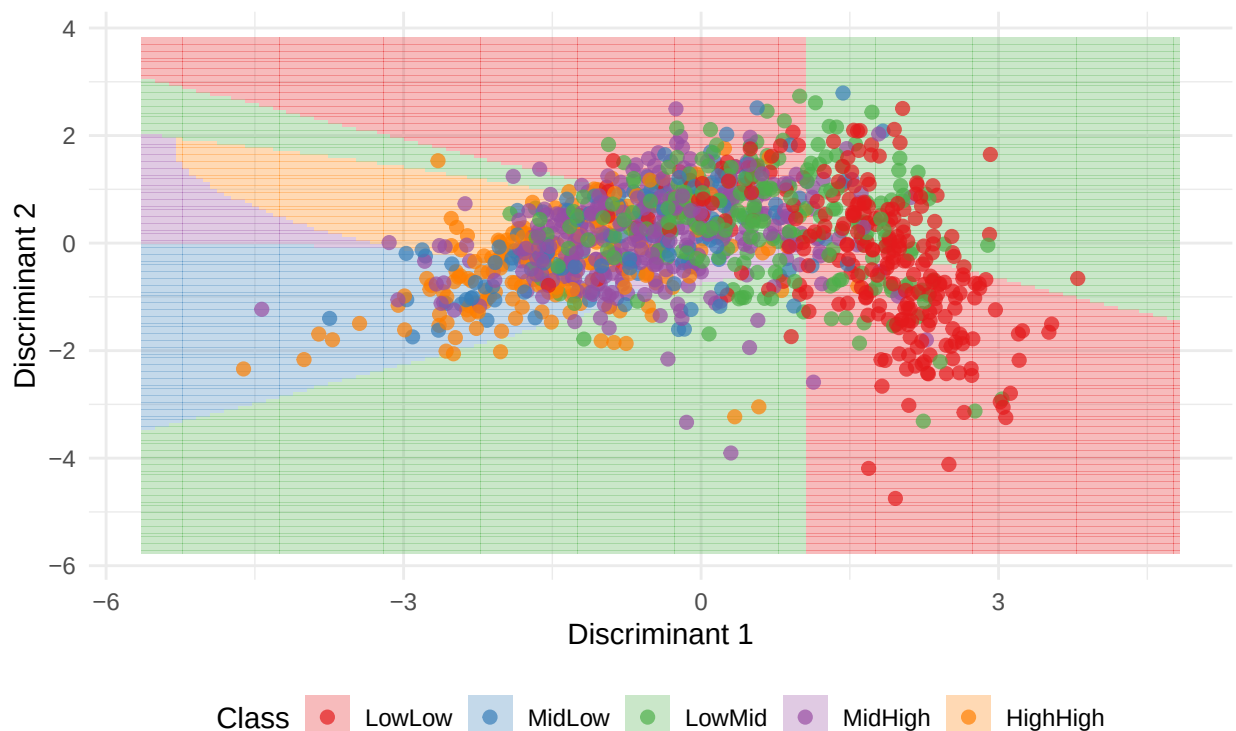
grid_pred <- predict(qda_fit, as.data.frame(grid_orig))$class
grid$class <- factor(grid_pred, levels = class_levels)

# Plot
ggplot() +
  geom_tile(data = grid, aes(x = LD1, y = LD2, fill = class), alpha = 0.3) +
  geom_point(data = qda_proj_train, aes(x = LD1, y = LD2, color = class), size = 2, alpha = 0.7) +
  scale_fill_manual(values = c("#E41A1C", "#377EB8", "#4DAF4A", "#984EA3", "#FF7F00"),
                    drop = FALSE) +
  scale_color_manual(values = c("#E41A1C", "#377EB8", "#4DAF4A", "#984EA3", "#FF7F00"),
                    drop = FALSE) +
  labs(title = "QDA Decision Regions for Combined Class",
       subtitle = paste0("Cross-validated Accuracy: ", round(max(qda_cv$results$Accuracy), 3)),
       x = "Discriminant 1",
       y = "Discriminant 2",
       fill = "Class",
       color = "Class") +
  theme_minimal() +
  theme(legend.position = "bottom",
        plot.title = element_text(face = "bold", size = 14),
        plot.subtitle = element_text(size = 11))

```

QDA Decision Regions for Combined Class

Cross-validated Accuracy: 0.449



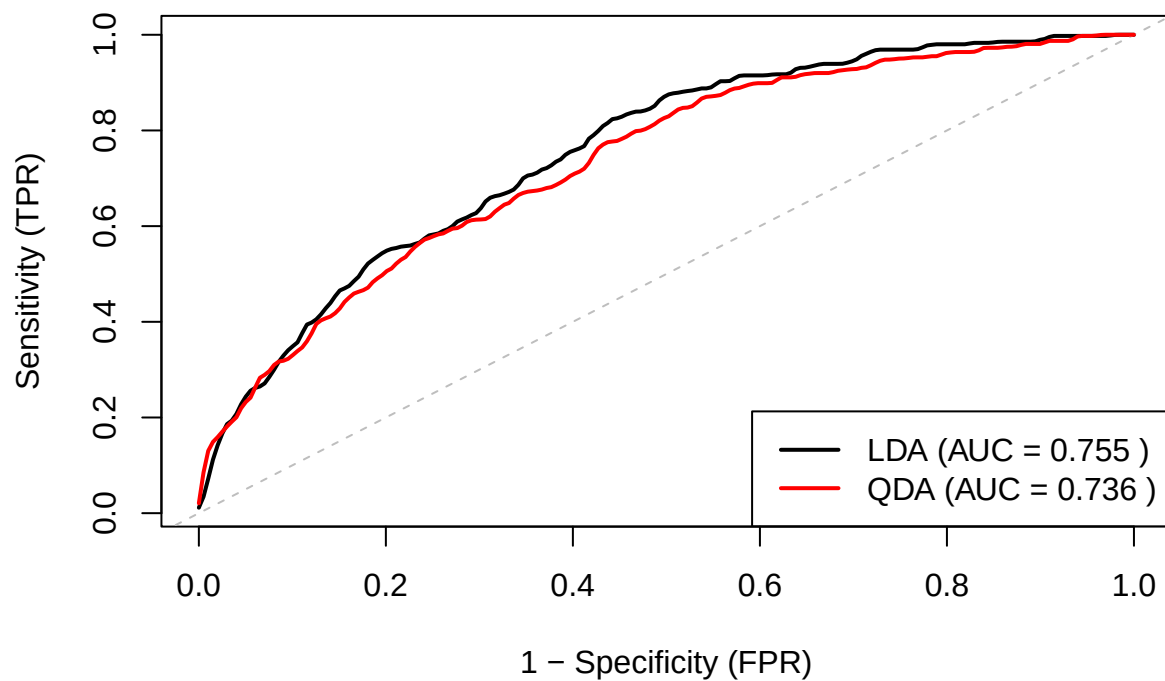
Comparison of LDA vs QDA

```
LDA_acc - QDA_acc
```

```
## [1] -0.01365188
```

```
# Graph AUC curves
plot(fpr_grid, macro_tpr_lda, type = "l", lwd = 2, col = "black",
     xlab = "1 - Specificity (FPR)", ylab = "Sensitivity (TPR)",
     main = paste("Macro-average ROC Curves of LDA and QDA"))
abline(a = 0, b = 1, lty = 2, col = "gray")
lines(fpr_grid, macro_tpr_qda, lwd = 2, col = "red")
legend("bottomright",
      legend = c(paste("LDA (AUC =", round(macro_auc_lda, 3), ")"),
                 paste("QDA (AUC =", round(macro_auc_qda, 3), ")")),
      col = c("black", "red"), lwd = 2)
```

Macro-average ROC Curves of LDA and QDA



LASSO with CV for best lambda

```
set.seed(67)
#initialize data in matrices
x_train <- as.matrix(train_data[, -ncol(train_data)])
```

```

y_train <- train_data$response
x_test  <- as.matrix(test_data[, -ncol(test_data)])
y_test  <- test_data$response

#run CV model
cv_lasso <- cv.glmnet(x_train, y_train,
                      family = "multinomial",
                      alpha = 1,
                      nfolds = 10)

plot(cv_lasso)

# Check optimal values of lambda
cat("Optimal lambda (min):", cv_lasso$lambda.min, "\n")

```

```
## Optimal lambda (min): 0.006424854
```

```
cat("Optimal lambda (1se):", cv_lasso$lambda.1se, "\n")
```

```
## Optimal lambda (1se): 0.02363305
```

```

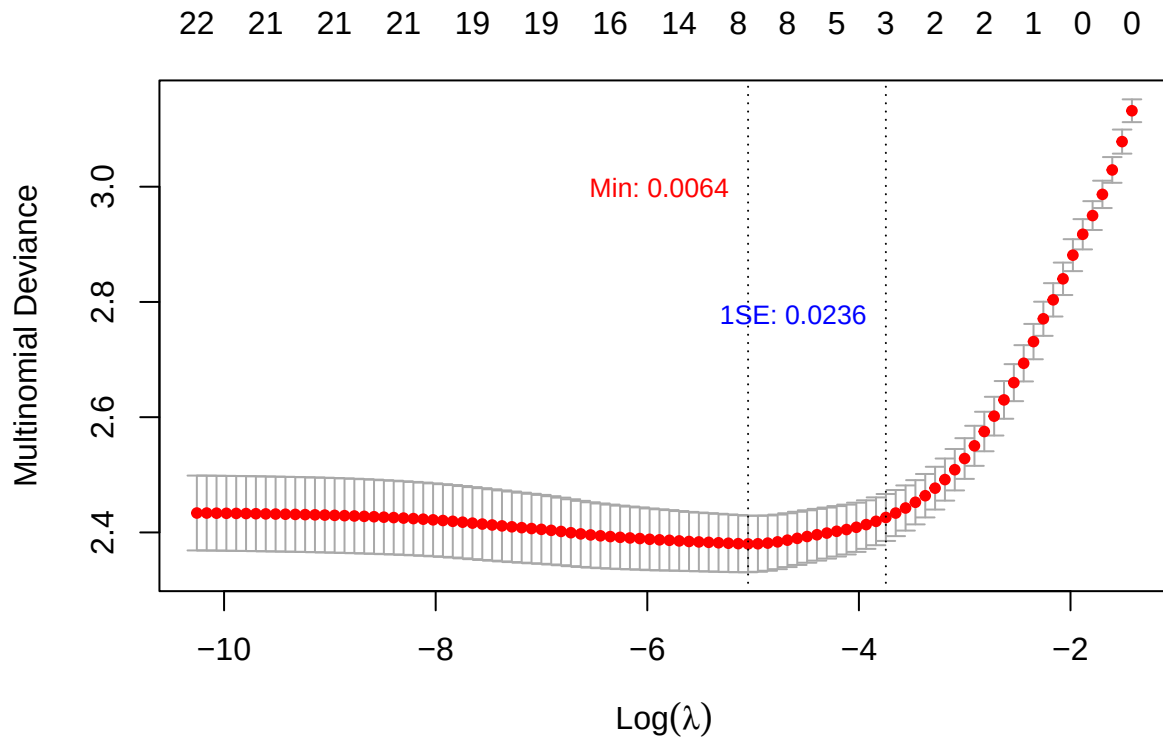
log_lambda_min <- log(cv_lasso$lambda.min)
log_lambda_1se <- log(cv_lasso$lambda.1se)

# Get the y-axis limits to position the labels
y_range <- range(cv_lasso$cvm + cv_lasso$cvstd, cv_lasso$cvm - cv_lasso$cvstd, na.rm = TRUE)

# Add labels near the top of the plot
text(x = log_lambda_min,
     y = y_range[2] * 0.95,
     labels = paste0("Min: ", round(cv_lasso$lambda.min, 4)),
     pos = 2,
     cex = 0.8,
     col = "red")

text(x = log_lambda_1se,
     y = y_range[2] * 0.88,
     labels = paste0("1SE: ", round(cv_lasso$lambda.1se, 4)),
     pos = 2,
     cex = 0.8,
     col = "blue")

```

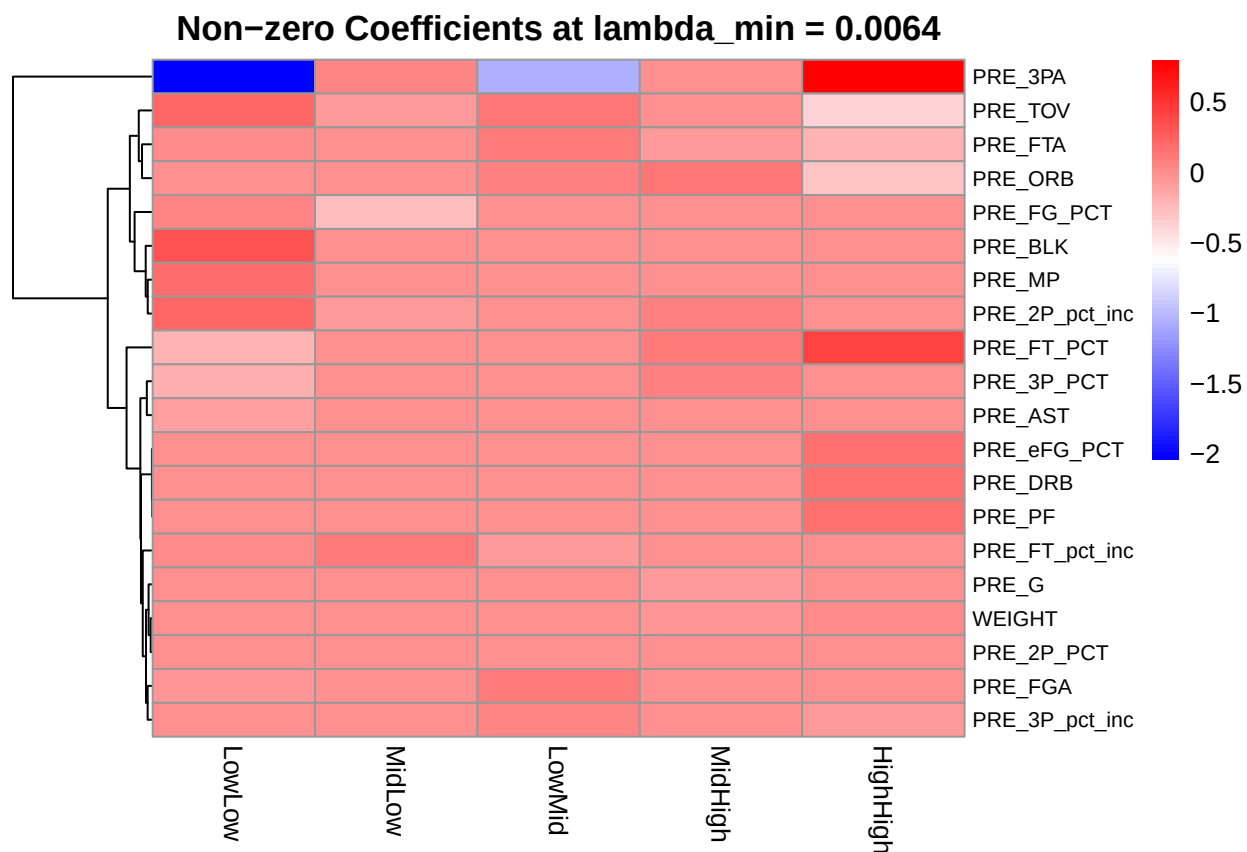


```
# Extract coefficients for lambda.min
coefs <- coef(cv_lasso, s = "lambda.min")

# Combine into a matrix (predictors x classes)
# Remove intercept row and convert to matrix
coef_matrix <- sapply(coefs, function(x) as.matrix(x)[-1, , drop = FALSE])
colnames(coef_matrix) <- levels(as.factor(y_train))
rownames(coef_matrix) <- rownames(coefs)[[1]][-1] # predictor names without intercept

# Keep only non-zero coefficients (or all if you prefer)
non_zero_rows <- apply(coef_matrix, 1, function(x) any(x != 0))
coef_nz <- coef_matrix[non_zero_rows, , drop = FALSE]

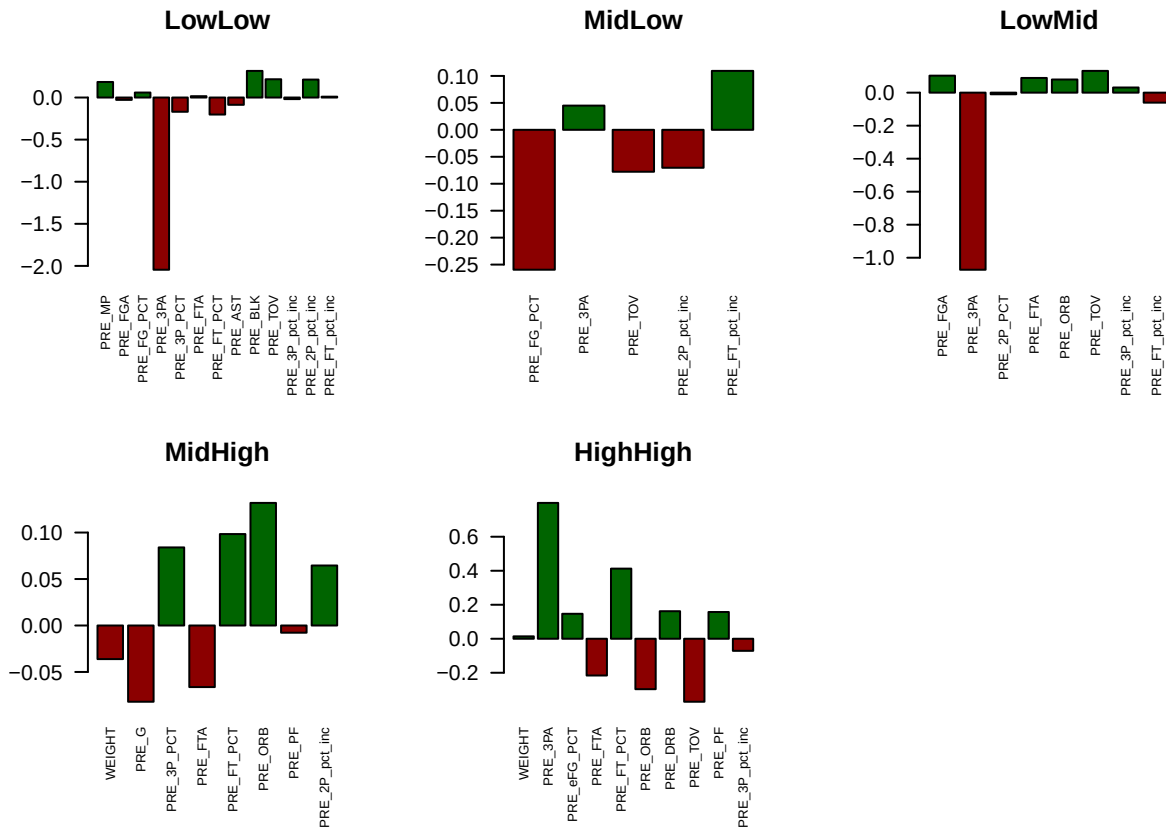
# Plot heatmap
pheatmap(coef_nz,
  cluster_rows = TRUE,
  cluster_cols = FALSE,
  main = paste0("Non-zero Coefficients at lambda_min = ", round(cv_lasso$lambda.min, 4)),
  color = colorRampPalette(c("blue", "white", "red"))(100),
  fontsize_row = 8,
  display_numbers = FALSE)
```



```

class_names <- levels(as.factor(y_train))
par(mfrow = c(2, 3)) # 2x3 grid for 5 classes + 1 empty
for (i in 1:5) {
  coef_vec <- as.matrix(coefs[[i]])[-1, ] # exclude intercept
  non_zero <- coef_vec != 0
  if (sum(non_zero) > 0) {
    barplot(coef_vec[non_zero],
            names.arg = rownames(coefs[[i]])[-1][non_zero],
            main = class_names[i],
            las = 2,
            cex.names = 0.6,
            col = ifelse(coef_vec[non_zero] > 0, "darkgreen", "darkred"))
  }
}
par(mfrow = c(1, 1)) # reset

```



```
# Calculate Test Accuracy
```

```
preds_min <- predict(cv_lasso, newx = x_test, s = "lambda.min", type = "class")
accuracy_min <- mean(as.character(preds_min) == as.character(y_test))
cat("Test accuracy (lambda.min):", accuracy_min, "\n")
```

```
## Test accuracy (lambda.min): 0.4778157
```

```
# Calculate CM
```

```
preds_min <- factor(preds_min, levels = levels(y_test))
conf_mat_min <- confusionMatrix(preds_min, y_test, mode = "prec_recall")
print(conf_mat_min)
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      48      2     16      10      2
##   MidLow       0      0      0       0      0
##   LowMid      10      1     13       6      0
##   MidHigh      8     20     31     50     20
##   HighHigh     0      8      0     19     29
```

```
##
```

```
## Overall Statistics
```

```
##
```

```
##           Accuracy : 0.4778
```

```
##          95% CI : (0.4194, 0.5367)
##    No Information Rate : 0.2901
##    P-Value [Acc > NIR] : 9.686e-12
##
##          Kappa : 0.3112
##
##    McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##          Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision          0.6154          NA          0.43333          0.3876
## Recall              0.7273          0.0000          0.21667          0.5882
## F1                  0.6667          NA          0.28889          0.4673
## Prevalence          0.2253          0.1058          0.20478          0.2901
## Detection Rate      0.1638          0.0000          0.04437          0.1706
## Detection Prevalence 0.2662          0.0000          0.10239          0.4403
## Balanced Accuracy    0.7976          0.5000          0.57185          0.6042
##
##          Class: HighHigh
## Precision          0.51786
## Recall              0.56863
## F1                  0.54206
## Prevalence          0.17406
## Detection Rate      0.09898
## Detection Prevalence 0.19113
## Balanced Accuracy    0.72853
```

```
# Calculate Class Error
```

```
class_error_min <- 1 - diag(conf_mat_min$table) / colSums(conf_mat_min$table)
print(class_error_min)
```

```
##    LowLow    MidLow    LowMid    MidHigh    HighHigh
## 0.2727273 1.0000000 0.7833333 0.4117647 0.4313725
```

```
# AUC Vectors
```

```
probs_min <- predict(cv_lasso, newx = x_test, s = "lambda.min", type = "response")
probs_min <- probs_min[,1]
colnames(probs_min) <- levels(y_test)
```

```
# Repeat
```

```
preds_1se <- predict(cv_lasso, newx = x_test, s = "lambda.1se", type = "class")
accuracy_1se <- mean(as.character(preds_1se) == as.character(y_test))
cat("Test accuracy (lambda.1se):", accuracy_1se, "\n")
```

```
## Test accuracy (lambda.1se): 0.4812287
```

```
preds_1se <- factor(preds_1se, levels = levels(y_test))
conf_mat_1se <- confusionMatrix(preds_1se, y_test, mode = "prec_recall")
print(conf_mat_1se)
```

```
## Confusion Matrix and Statistics
##
```

```
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      51      2      17      11      2
##   MidLow       0      0       0       0      0
##   LowMid       5      1      10       6      1
##   MidHigh     10     23     33     55     23
##   HighHigh      0      5       0     13     25
##
## Overall Statistics
##
##           Accuracy : 0.4812
##           95% CI : (0.4228, 0.5401)
##   No Information Rate : 0.2901
##   P-Value [Acc > NIR] : 4.251e-12
##
##           Kappa : 0.3101
##
## McNemar's Test P-Value : 1.472e-09
##
## Statistics by Class:
##
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision           0.6145           NA           0.43478           0.3819
## Recall              0.7727           0.0000           0.16667           0.6471
## F1                  0.6846           NA           0.24096           0.4803
## Prevalence          0.2253           0.1058           0.20478           0.2901
## Detection Rate      0.1741           0.0000           0.03413           0.1877
## Detection Prevalence 0.2833           0.0000           0.07850           0.4915
## Balanced Accuracy    0.8159           0.5000           0.55544           0.6096
##
##           Class: HighHigh
## Precision           0.58140
## Recall              0.49020
## F1                  0.53191
## Prevalence          0.17406
## Detection Rate      0.08532
## Detection Prevalence 0.14676
## Balanced Accuracy    0.70791
```

```
class_error_1se <- 1 - diag(conf_mat_1se$table) / colSums(conf_mat_1se$table)
print(class_error_1se)
```

```
##   LowLow   MidLow   LowMid   MidHigh   HighHigh
## 0.2272727 1.0000000 0.8333333 0.3529412 0.5098039
```

```
probs_1se <- predict(cv_lasso, newx = x_test, s = "lambda.1se", type = "response")
probs_1se <- probs_1se[,1]
colnames(probs_1se) <- levels(y_test)
```

```
# Load model results (done for every model)
model_results <- rbind(model_results, data.frame(
  Model = "LASSO Min Lambda",
  Accuracy = accuracy_min,
  Precision = mean(conf_mat_min$byClass[, "Precision"], na.rm = TRUE),
```

```

Recall = mean(conf_mat_min$byClass[, "Recall"], na.rm = TRUE),
F1 = mean(conf_mat_min$byClass[, "F1"], na.rm = TRUE),
AUC = macro_auc(y_test, probs_min)
))

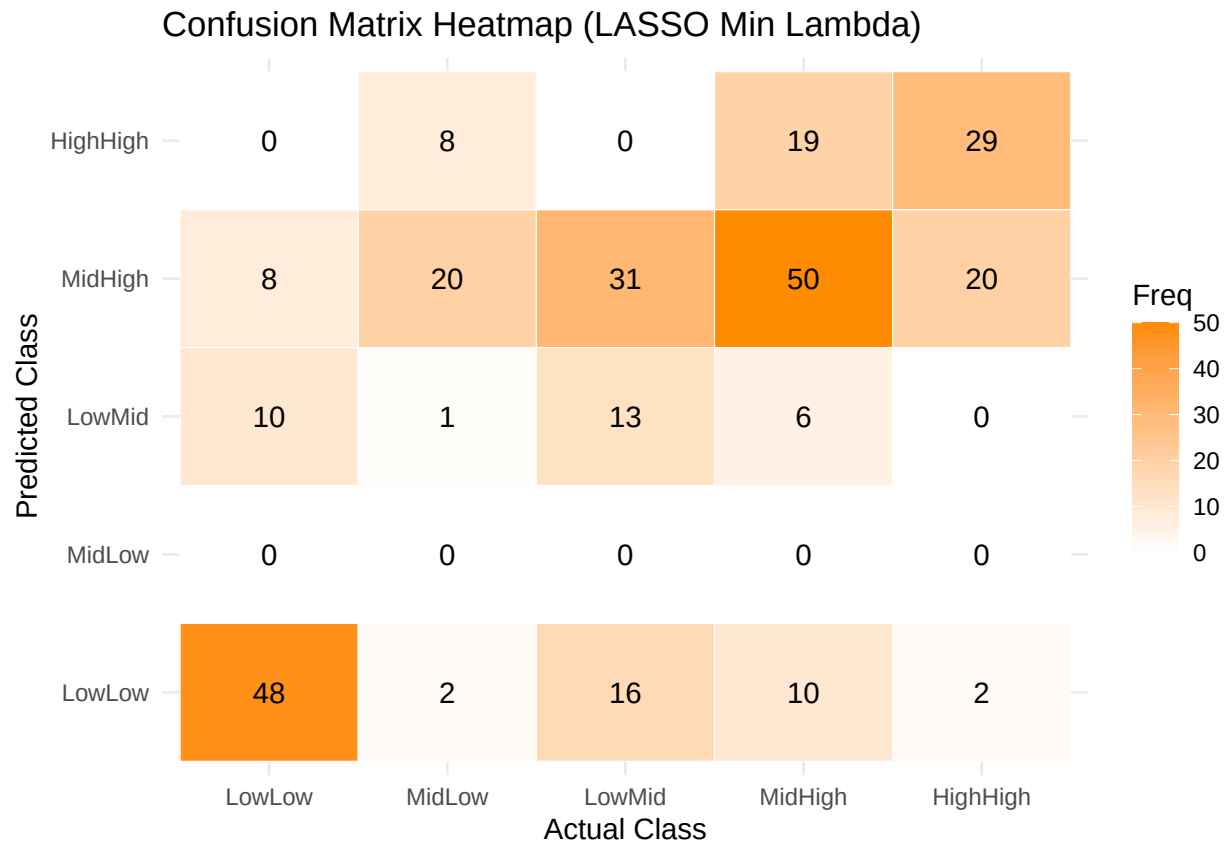
model_results <- rbind(model_results, data.frame(
  Model = "LASSO 1se Lambda",
  Accuracy = accuracy_1se,
  Precision = mean(conf_mat_1se$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(conf_mat_1se$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(conf_mat_1se$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(y_test, probs_1se)
))

```

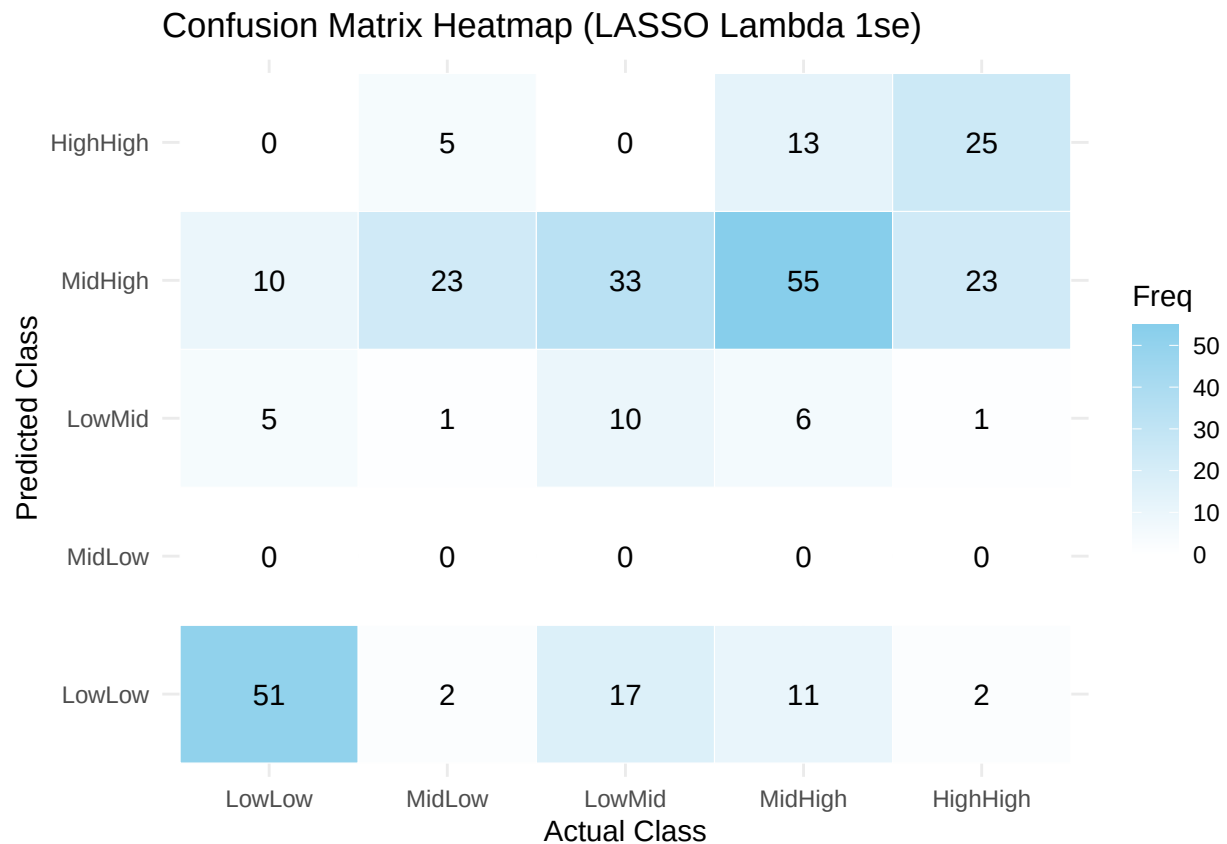
```

# Create confusion matrix (done for every model)
cm_table <- as.data.frame(conf_mat_min$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (LASSO Min Lambda)",
       x = "Actual Class", y = "Predicted Class")

```



```
cm_table <- as.data.frame(conf_mat_1se$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (LASSO Lambda 1se)",
       x = "Actual Class", y = "Predicted Class")
```



PCA

```
# Set PCA statistics
pca_result <- prcomp(df_scaled[, -ncol(df_scaled)], center = TRUE, scale. = TRUE)
explained_var <- summary(pca_result)$importance[2, ]
num_pcs <- which(cumsum(explained_var) >= 0.9)[1]

# Set PCA dataframe
df_pca <- data.frame(pca_result$x[, 1:num_pcs], response = df_scaled$response)
train_data_pca <- df_pca[train_index, ]
test_data_pca <- df_pca[-train_index, ]

# PCA with CV
train_control <- trainControl(method = "cv", number = 10,
```

```

summaryFunction = multiClassSummary)

model_pca_cv <- train(response ~ ., data = train_data_pca,
  method = "multinom",
  trControl = train_control,
  trace = FALSE)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

print(model_pca_cv)

## Penalized Multinomial Regression
##
## 1186 samples
## 13 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1067, 1067, 1066, 1068, ...
## Resampling results across tuning parameters:
##
## decay Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 0e+00 0.4932268 0.3334015 NaN 0.4315010 0.8656378
## 1e-04 0.4932268 0.3334015 NaN 0.4315010 0.8656378
## 1e-01 0.4932197 0.3333812 NaN 0.4314369 0.8656285
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.4072215 0.8702628 0.4072215 0.4315010
## 0.4072215 0.8702628 0.4072215 0.4315010
## 0.4072215 0.8703048 0.4072215 0.4314369
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.09864536 0.6485694
## 0.09864536 0.6485694
## 0.09864394 0.6485327
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was decay = 1e-04.

cat("Cross-validated Accuracy (training set):", max(model_pca_cv$results$Accuracy), "\n")

## Cross-validated Accuracy (training set): 0.4932268

# Set our fit model
model_pca <- multinom(response ~ ., data = train_data_pca, trace = FALSE)

preds_pca <- predict(model_pca, newdata = test_data_pca)
preds_pca <- factor(preds_pca, levels = levels(test_data_pca$response))

# Calc CM
cm_pca <- confusionMatrix(preds_pca, test_data_pca$response, mode = "prec_recall")
print(cm_pca)

```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      46      2      11      9      0
##   MidLow       0      0       0      0      0
##   LowMid      10      2      15      7      3
##   MidHigh     10     20      34     44     20
##   HighHigh      0      7       0     25     28
##
## Overall Statistics
##
##           Accuracy : 0.4539
##           95% CI : (0.3959, 0.5129)
##   No Information Rate : 0.2901
##   P-Value [Acc > NIR] : 2.123e-09
##
##           Kappa : 0.281
##
##   McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision           0.6765           NA           0.40541           0.3438
## Recall              0.6970           0.0000           0.25000           0.5176
## F1                  0.6866           NA           0.30928           0.4131
## Prevalence          0.2253           0.1058           0.20478           0.2901
## Detection Rate      0.1570           0.0000           0.05119           0.1502
## Detection Prevalence 0.2321           0.0000           0.12628           0.4369
## Balanced Accuracy    0.8000           0.5000           0.57779           0.5569
##
##           Class: HighHigh
## Precision           0.46667
## Recall              0.54902
## F1                  0.50450
## Prevalence          0.17406
## Detection Rate      0.09556
## Detection Prevalence 0.20478
## Balanced Accuracy    0.70839
```

```
test_acc <- mean(as.character(preds_pca) == as.character(test_data_pca$response))
cat("Test Accuracy (PCA-based model):", test_acc, "\n")
```

```
## Test Accuracy (PCA-based model): 0.4539249
```

```
loadings <- pca_result$rotation[, 1:num_pcs]
print("PCA Loadings (contribution of original variables):")
```

```
## [1] "PCA Loadings (contribution of original variables):"
```

```
print(loadings)
```

##		PC1	PC2	PC3	PC4	PC5
##	WEIGHT	0.310576735	-0.11600745	0.022405694	-0.090180825	0.01486769
##	HEIGHT_INCHES	0.306018471	-0.12869752	-0.071442881	-0.070894339	0.04853623
##	PRE_G	-0.091069205	-0.39616697	0.179522649	-0.150512294	-0.04018886
##	PRE_GS	-0.108814951	-0.33153667	0.243856055	-0.150788708	-0.11745689
##	PRE_MP	-0.131801927	-0.34671668	0.252903108	-0.147007736	-0.06628751
##	PRE_FGA	-0.060335081	0.20465217	0.167377182	-0.448716734	0.07429732
##	PRE_FG_PCT	0.297628706	-0.08839393	0.066116020	-0.009784959	-0.09895303
##	PRE_3PA	-0.304268051	0.10539397	-0.006859503	-0.100076805	0.11678585
##	PRE_3P_PCT	-0.209899910	0.05912609	0.069077397	-0.027605119	-0.03071896
##	PRE_2PA	0.163609176	0.25793448	0.356691231	-0.006040583	-0.13122173
##	PRE_2P_PCT	0.092818662	0.24657497	0.352240719	0.283858437	-0.18429705
##	PRE_eFG_PCT	0.065374880	0.25458067	0.359438727	0.286058254	-0.16684636
##	PRE_FTA	0.170662156	0.11379092	0.164509523	-0.343449549	-0.08557226
##	PRE_FT_PCT	-0.247252477	0.07428605	0.092619611	-0.120717187	0.02573812
##	PRE_ORB	0.252306293	-0.01153781	0.173584896	-0.046970034	0.35464067
##	PRE_DRB	0.193106037	-0.03361403	-0.099911399	-0.107603026	-0.37179356
##	PRE_TRB	0.236534587	-0.06157029	-0.102453639	-0.088923308	-0.45323070
##	PRE_AST	-0.235526532	0.03941034	-0.091674798	-0.001542908	-0.37897486
##	PRE_STL	-0.150644666	0.00788266	-0.186349012	-0.042236917	-0.47896250
##	PRE_BLK	0.306173178	-0.04844280	-0.014637065	0.030408065	-0.02030777
##	PRE_TOV	0.016687285	0.09544624	0.009094554	-0.275116011	-0.08817865
##	PRE_PF	0.217643408	-0.16667332	-0.183079463	-0.046787190	0.04078897
##	PRE_PTS	0.038980952	0.20570127	0.223216572	-0.460695696	0.01450406
##	PRE_FG_pct_inc	0.004719971	-0.32262204	0.195555697	0.068318283	0.01528496
##	PRE_3P_pct_inc	-0.185918557	-0.15406746	0.155208206	-0.022473690	-0.07223330
##	PRE_2P_pct_inc	-0.029798898	-0.10309029	0.331152399	0.300150507	-0.04396151
##	PRE_FT_pct_inc	-0.012801073	-0.28517770	0.189787389	0.053510073	-0.02871863
##		PC6	PC7	PC8	PC9	PC10
##	WEIGHT	-0.102440463	0.01945161	-0.10761587	0.0441645755	0.071696264
##	HEIGHT_INCHES	-0.197861654	0.03426717	-0.07151755	0.0003343841	0.104053219
##	PRE_G	-0.003096375	-0.22717250	-0.03676991	-0.0148742508	-0.046949635
##	PRE_GS	0.016534577	-0.33539249	-0.03253507	0.0190600895	-0.111168430
##	PRE_MP	0.015534053	-0.28855828	-0.01783077	-0.0040057105	-0.074098536
##	PRE_FGA	-0.100342569	0.14375073	0.12366593	0.3050992422	0.010059538
##	PRE_FG_PCT	-0.004409603	-0.13322309	-0.23111057	-0.0781484906	0.275050775
##	PRE_3PA	-0.254619740	0.10951615	0.04972839	0.1597272981	-0.063145956
##	PRE_3P_PCT	-0.296272720	0.12555504	-0.25505483	-0.6921348003	0.238149213
##	PRE_2PA	0.100909258	-0.04779157	0.12626039	0.0630074925	0.022293237
##	PRE_2P_PCT	-0.025008678	-0.09948989	0.03965274	-0.0408271040	0.035244571
##	PRE_eFG_PCT	-0.049976061	-0.09752382	0.01237576	-0.0373396158	0.032764705
##	PRE_FTA	0.205143540	0.04317515	-0.18480518	-0.1013142886	0.019922747
##	PRE_FT_PCT	-0.261178994	0.02336108	-0.33828357	-0.1089984569	-0.003053189
##	PRE_ORB	0.124199677	-0.04014109	0.07343547	-0.1625242423	0.025815599
##	PRE_DRB	-0.345438973	0.08949877	0.25515900	-0.1880550171	-0.431033802
##	PRE_TRB	-0.277619833	0.07917485	0.06957106	0.0551909921	-0.111570151
##	PRE_AST	0.310881381	-0.06803405	-0.13439517	-0.0227785634	0.035482104
##	PRE_STL	0.158545817	0.08979451	-0.15790259	0.2435688666	0.381844051
##	PRE_BLK	-0.036114530	-0.04979654	-0.06310273	0.0026893567	0.123028939
##	PRE_TOV	0.531394851	0.18724542	0.12860582	-0.3867088692	-0.291267960
##	PRE_PF	0.152259628	0.28034979	-0.09726541	-0.0614783636	0.162613277
##	PRE_PTS	-0.109213441	0.11347422	-0.11912295	0.1513782274	0.160113006
##	PRE_FG_pct_inc	0.044722428	0.45395118	-0.24968559	0.1021009545	-0.201073762
##	PRE_3P_pct_inc	-0.075108836	0.27114166	0.49431413	-0.1656599680	0.269992950

```
## PRE_2P_pct_inc 0.037758904 0.43277509 -0.26247403 0.1596732804 -0.230500277
## PRE_FT_pct_inc 0.045372377 0.19061518 0.37416558 0.0154122996 0.399782204
##          PC11          PC12          PC13
## WEIGHT      0.058519996 0.09333076 -0.028184110
## HEIGHT_INCHES -0.011694787 0.17386310 0.076222008
## PRE_G        0.002715547 0.13765013 0.051480749
## PRE_GS       0.027568844 0.14322979 0.024691520
## PRE_MP       0.028870044 0.11442966 0.009178278
## PRE_FGA      -0.076756280 0.25203787 -0.104767828
## PRE_FG_PCT   0.056708376 -0.27041638 -0.010116838
## PRE_3PA      -0.118367878 0.23071748 0.168132304
## PRE_3P_PCT   -0.132992750 0.28816877 -0.346557121
## PRE_2PA      0.028751024 0.15501850 -0.076811133
## PRE_2P_PCT   0.016604797 0.09011226 0.147592764
## PRE_eFG_PCT  -0.009846565 0.09309186 0.167533579
## PRE_FTA      0.079548232 -0.39511675 -0.123032661
## PRE_FT_PCT   -0.005951696 -0.32729026 0.607281583
## PRE_ORB      0.095369951 0.06374772 -0.145015816
## PRE_DRB      -0.075628303 -0.09555228 -0.006762200
## PRE_TRB      -0.025164286 -0.02048343 -0.052993184
## PRE_AST      -0.058816769 -0.03062988 0.009038546
## PRE_STL      0.101423145 0.20489575 -0.103729875
## PRE_BLK      -0.055082361 0.25445469 0.332816936
## PRE_TOV      -0.140123639 0.15500691 0.244921273
## PRE_PF       0.066511609 0.31256945 0.375978211
## PRE_PTS      -0.013609522 -0.08498533 0.001161631
## PRE_FG_pct_inc 0.078072671 -0.08208679 -0.156071581
## PRE_3P_pct_inc 0.646933312 -0.10277365 0.122327463
## PRE_2P_pct_inc -0.024377950 0.07019345 -0.046466246
## PRE_FT_pct_inc -0.681751633 -0.24304954 0.051015208
```

```
# Run it
train_control <- trainControl(method = "cv", number = 10,
                              summaryFunction = multiClassSummary)

model_pca_cv <- train(response ~ ., data = train_data_pca,
                      method = "multinom",
                      trControl = train_control,
                      trace = FALSE)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
cat("Cross-validated accuracy (PCA-based model, training set):",
    max(model_pca_cv$results$Accuracy), "\n")
```

```
## Cross-validated accuracy (PCA-based model, training set): 0.4990444
```

```
model_pca <- multinom(response ~ ., data = train_data_pca, trace = FALSE)

preds_pca <- predict(model_pca, newdata = test_data_pca)
preds_pca <- factor(preds_pca, levels = levels(test_data_pca$response))
```

```

cm_pca <- confusionMatrix(preds_pca, test_data_pca$response, mode = "prec_recall")
print(cm_pca)

## Confusion Matrix and Statistics
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
## LowLow      46      2      11       9       0
## MidLow       0      0       0       0       0
## LowMid      10      2      15       7       3
## MidHigh     10     20      34      44      20
## HighHigh     0      7       0      25      28
##
## Overall Statistics
##
##           Accuracy : 0.4539
##           95% CI : (0.3959, 0.5129)
## No Information Rate : 0.2901
## P-Value [Acc > NIR] : 2.123e-09
##
##           Kappa : 0.281
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision           0.6765           NA           0.40541           0.3438
## Recall              0.6970           0.0000           0.25000           0.5176
## F1                  0.6866           NA           0.30928           0.4131
## Prevalence          0.2253           0.1058           0.20478           0.2901
## Detection Rate      0.1570           0.0000           0.05119           0.1502
## Detection Prevalence 0.2321           0.0000           0.12628           0.4369
## Balanced Accuracy    0.8000           0.5000           0.57779           0.5569
##
##           Class: HighHigh
## Precision           0.46667
## Recall              0.54902
## F1                  0.50450
## Prevalence          0.17406
## Detection Rate      0.09556
## Detection Prevalence 0.20478
## Balanced Accuracy    0.70839

test_acc_pca <- mean(as.character(preds_pca) == as.character(test_data_pca$response))
cat("Test Accuracy (PCA-based model):", test_acc_pca, "\n")

## Test Accuracy (PCA-based model): 0.4539249

probs_pca <- predict(model_pca, newdata = test_data_pca, type = "probs")
probs_pca <- as.matrix(probs_pca)

```

```

colnames(probs_pca) <- colnames(probs_pca)
probs_pca <- probs_pca[, levels(test_data_pca$response), drop = FALSE]

# Use first 2 PCs from your pca_result (even if you used more in model)
pca_scores_train <- data.frame(
  PC1 = pca_result$x[train_index, 1],
  PC2 = pca_result$x[train_index, 2],
  response = df_scaled$response[train_index],
  Dataset = "Train"
)

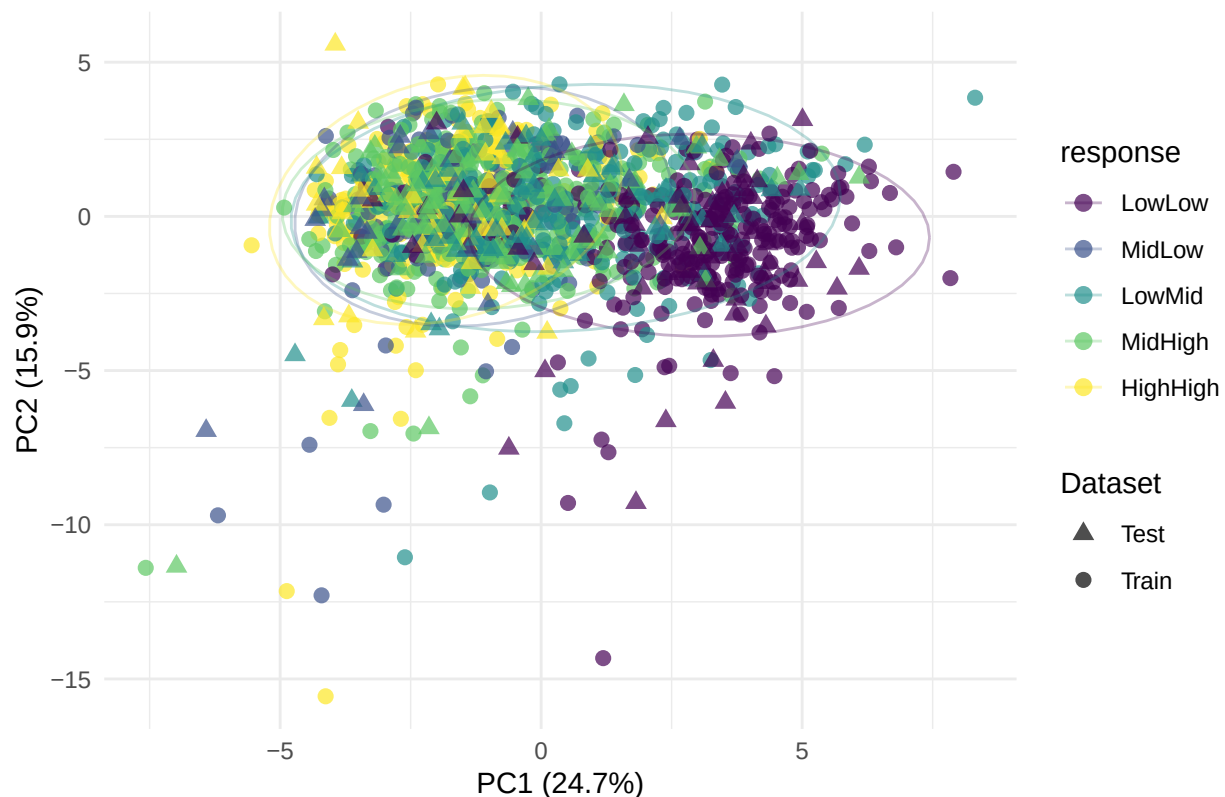
pca_scores_test <- data.frame(
  PC1 = pca_result$x[-train_index, 1],
  PC2 = pca_result$x[-train_index, 2],
  response = df_scaled$response[-train_index],
  Dataset = "Test"
)

pca_scores_all <- rbind(pca_scores_train, pca_scores_test)
var_pc1 <- round(explained_var[1] * 100, 1)
var_pc2 <- round(explained_var[2] * 100, 1)

ggplot(pca_scores_all, aes(x = PC1, y = PC2, color = response, shape = Dataset)) +
  geom_point(size = 2.5, alpha = 0.7) +
  stat_ellipse(data = subset(pca_scores_all, Dataset == "Train"),
    aes(group = response), level = 0.95, alpha = 0.3) +
  theme_minimal() +
  labs(title = "PCA Score Plot - First Two Principal Components",
    x = paste0("PC1 (", var_pc1, "%)"),
    y = paste0("PC2 (", var_pc2, "%)")) +
  scale_shape_manual(values = c("Train" = 16, "Test" = 17))

```

PCA Score Plot – First Two Principal Components

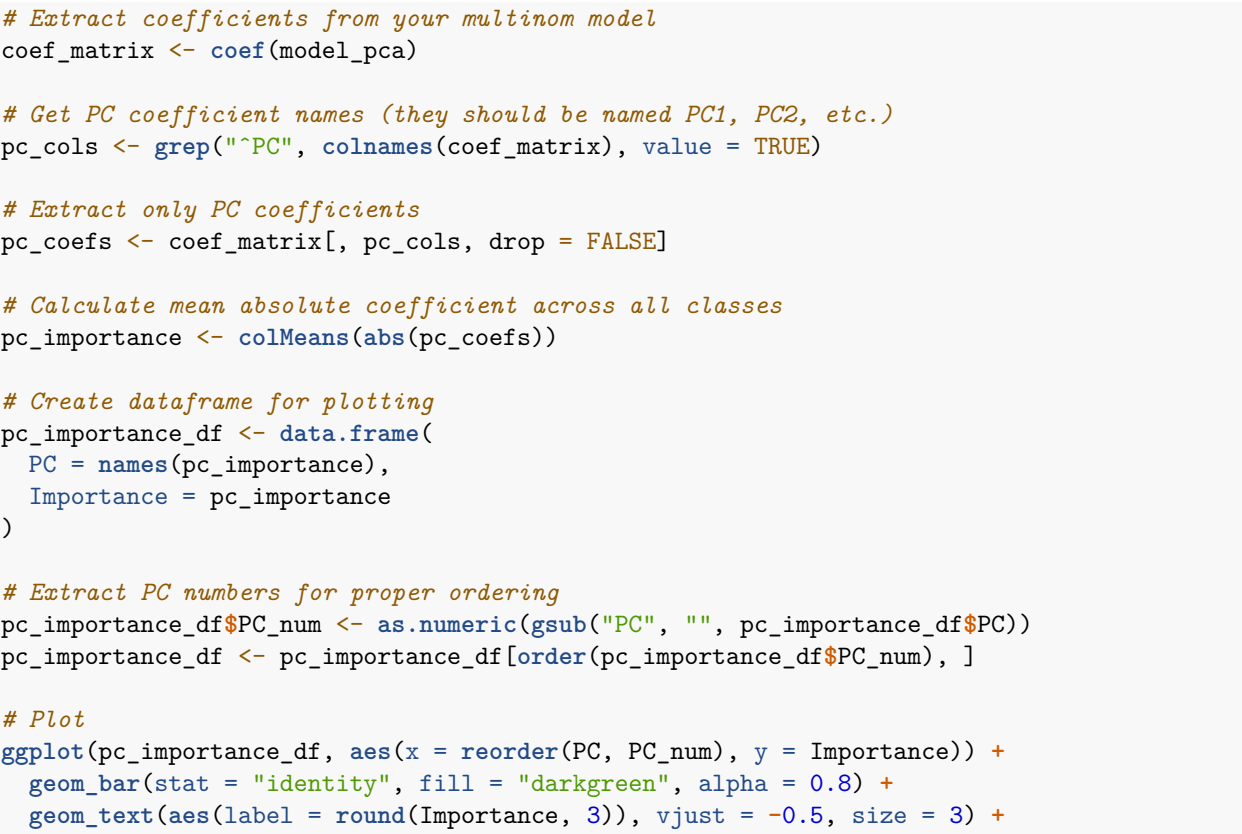


```
# Extract loadings for first 2 PCs
loadings_df <- as.data.frame(pca_result$rotation[, 1:2])
loadings_df$feature <- rownames(loadings_df)
loadings_df$importance <- sqrt(loadings_df$PC1^2 + loadings_df$PC2^2)

# Select top features to label (avoid overcrowding)
loadings_top <- loadings_df[order(loadings_df$importance, decreasing = TRUE)[1:min(15, nrow(loadings_df))]]

ggplot(loadings_df, aes(x = PC1, y = PC2)) +
  geom_segment(aes(xend = 0, yend = 0), alpha = 0.3, color = "gray60") +
  geom_point(alpha = 0.3) +
  geom_text(data = loadings_top,
            aes(label = feature, color = importance),
            size = 3, fontface = "bold") +
  scale_color_gradient(low = "darkblue", high = "darkred") +
  theme_minimal() +
  labs(title = "PCA Loadings - Feature Contributions to PC1 and PC2",
       subtitle = paste0("Top ", nrow(loadings_top), " features labeled by importance"),
       x = paste0("PC1 (", var_pc1, "%)"),
       y = paste0("PC2 (", var_pc2, "%)"),
       color = "Importance") +
  geom_hline(yintercept = 0, linetype = "dashed", alpha = 0.5) +
  geom_vline(xintercept = 0, linetype = "dashed", alpha = 0.5) +
  theme(legend.position = "bottom")
```

Top 15 features labeled by importance



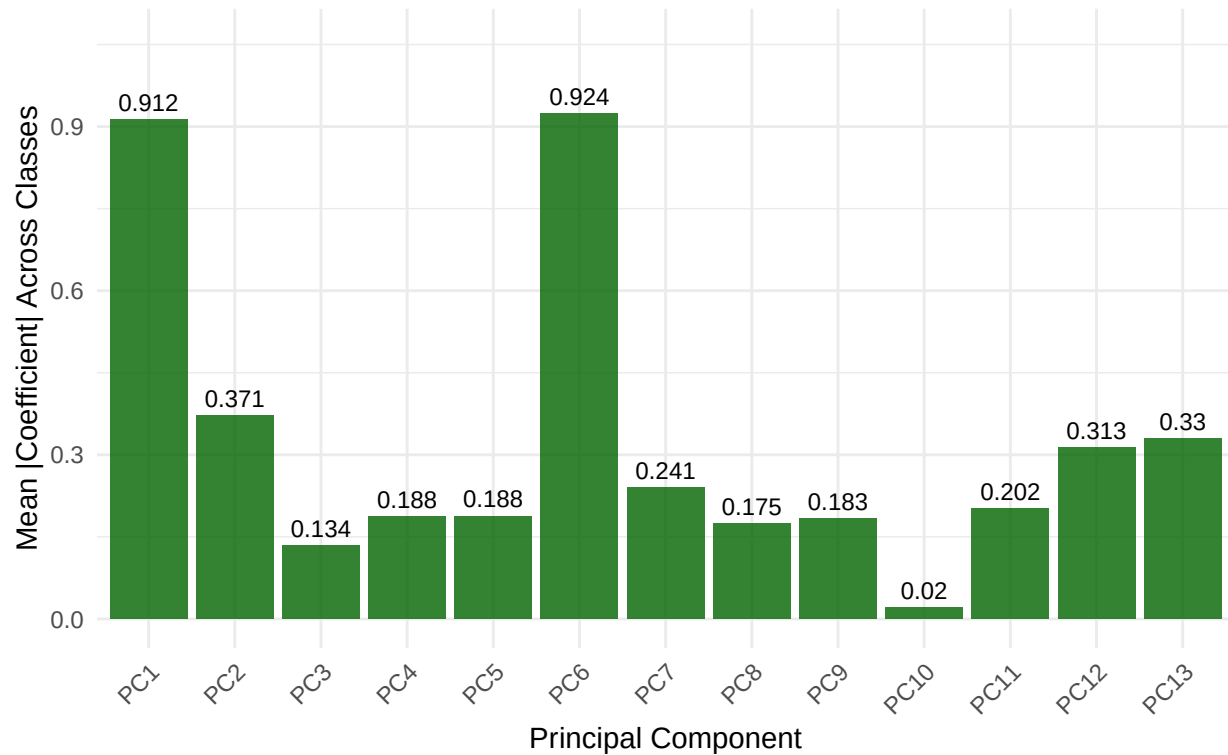
```

theme_minimal() +
labs(title = "Average Importance of PCs in Multinomial Model",
      subtitle = paste0("Based on mean absolute coefficients across all classes (",
                          num_pcs, " PCs used)"),
      x = "Principal Component",
      y = "Mean |Coefficient| Across Classes") +
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
expand_limits(y = max(pc_importance_df$Importance) * 1.15)

```

Average Importance of PCs in Multinomial Model

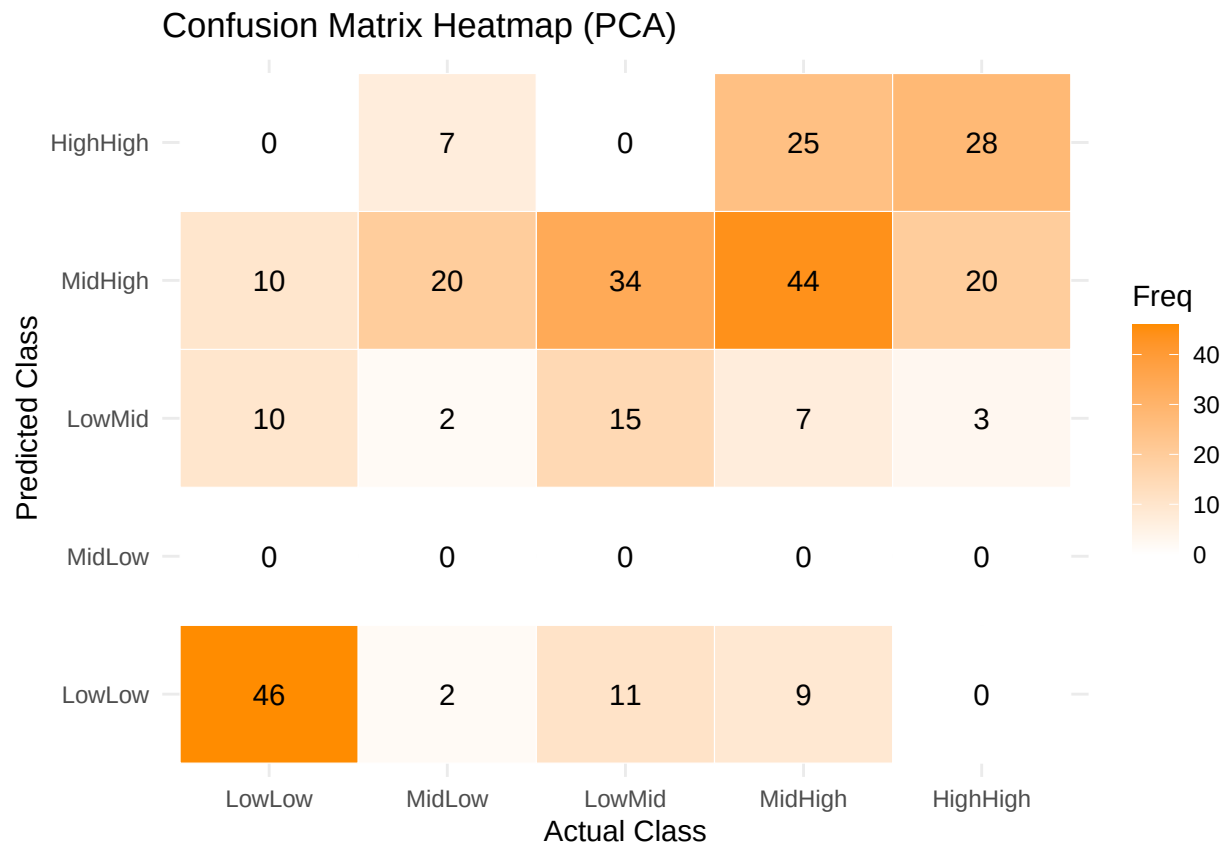
Based on mean absolute coefficients across all classes (13 PCs used)



```

cm_table <- as.data.frame(cm_pca$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (PCA)",
       x = "Actual Class", y = "Predicted Class")

```



```
model_results <- rbind(model_results, data.frame(
  Model = "PCA",
  Accuracy = test_acc_pca,
  Precision = mean(cm_pca$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_pca$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_pca$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data_pca$response, probs_pca)
))
```

SVM Radial and Linear

```
set.seed(67)
# CV SVM
svm_linear <- train(
  response ~ .,
  data = train_data,
  method = "svmLinear2",
  trControl = trainControl(
    method = "cv",
    number = 10,
    classProbs = TRUE,
    summaryFunction = multiClassSummary
  ),
  preProcess = c("center", "scale", "pca"),
```

```
tuneGrid = expand.grid(cost = c(0.01, 0.1, 1, 10)),
  probability = TRUE
)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
print(svm_linear)
```

```
## Support Vector Machines with Linear Kernel
##
## 1186 samples
## 27 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## Pre-processing: centered (27), scaled (27), principal component
## signal extraction (27)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1067, 1068, 1068, 1067, 1066, ...
## Resampling results across tuning parameters:
##
## cost logLoss AUC prAUC Accuracy Kappa Mean_F1
## 0.01 1.220661 0.7559764 0.4257302 0.4645095 0.2767078 NaN
## 0.10 1.209106 0.7598262 0.4375248 0.4932324 0.3312741 NaN
## 1.00 1.210306 0.7616912 0.4392217 0.4941157 0.3337833 NaN
## 10.00 1.210145 0.7627623 0.4417608 0.4949484 0.3351066 NaN
## Mean_Sensitivity Mean_Specificity Mean_Pos_Pred_Value Mean_Neg_Pred_Value
## 0.3780854 0.8537640 NaN 0.8692518
## 0.4284745 0.8650533 NaN 0.8703919
## 0.4312712 0.8656352 NaN 0.8704900
## 0.4325163 0.8658924 NaN 0.8706191
## Mean_Precision Mean_Recall Mean_Detection_Rate Mean_Balanced_Accuracy
## NaN 0.3780854 0.09290189 0.6159247
## NaN 0.4284745 0.09864648 0.6467639
## NaN 0.4312712 0.09882314 0.6484532
## NaN 0.4325163 0.09898969 0.6492043
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was cost = 10.
```

```
cat("Best cost (C):", svm_linear$bestTune$C, "\n")
```

```
## Best cost (C):
```

```
cat("Cross-validated accuracy (training set):", max(svm_linear$results$Accuracy), "\n")
```

```
## Cross-validated accuracy (training set): 0.4949484
```

```
preds_svm_linear <- predict(svm_linear, newdata = test_data)
preds_svm_linear <- factor(preds_svm_linear, levels = levels(test_data$response))
```

```
cm_svm_linear <- confusionMatrix(preds_svm_linear, test_data$response, mode = "prec_recall")
print(cm_svm_linear)
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
## LowLow      52      2      10      9      0
## MidLow       0      0       0       0      0
## LowMid       4      2      23      10     4
## MidHigh      10     23      27      47     16
## HighHigh     0      4       0      19     31
##
```

```
## Overall Statistics
```

```
##
##           Accuracy : 0.5222
##           95% CI : (0.4633, 0.5806)
## No Information Rate : 0.2901
## P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.3713
##
## McNemar's Test P-Value : NA
##
```

```
## Statistics by Class:
```

```
##
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision           0.7123           NA           0.5349           0.3821
## Recall              0.7879           0.0000           0.3833           0.5529
## F1                  0.7482           NA           0.4466           0.4519
## Prevalence          0.2253           0.1058           0.2048           0.2901
## Detection Rate      0.1775           0.0000           0.0785           0.1604
## Detection Prevalence 0.2491           0.0000           0.1468           0.4198
## Balanced Accuracy    0.8477           0.5000           0.6487           0.5938
##
##           Class: HighHigh
## Precision           0.5741
## Recall              0.6078
## F1                  0.5905
## Prevalence          0.1741
## Detection Rate      0.1058
## Detection Prevalence 0.1843
## Balanced Accuracy    0.7564
```

```
test_acc_linear <- mean(as.character(preds_svm_linear) == as.character(test_data$response))
cat("Test Accuracy (SVM Linear):", test_acc_linear, "\n")
```

```
## Test Accuracy (SVM Linear): 0.5221843
```

```
probs_svm_linear <- predict(svm_linear, newdata = test_data, type = "prob")
probs_svm_linear <- as.matrix(probs_svm_linear)
```

```
colnames(probs_svm_linear) <- colnames(probs_svm_linear)
```

```

probs_svm_linear <- probs_svm_linear[, levels(test_data_pca$response), drop = FALSE]

model_results <- rbind(model_results, data.frame(
  Model = "SVM Linear",
  Accuracy = test_acc_linear,
  Precision = mean(cm_svm_linear$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_svm_linear$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_svm_linear$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_svm_linear)
))

```

```

# Get variable importance
var_imp <- varImp(svm_linear, scale = FALSE)

# Extract importance based on its structure
imp_values <- var_imp$importance

# Extract feature names and scores properly
if(is.matrix(imp_values) || is.data.frame(imp_values)) {
  # If multiple columns (multiclass), take row means
  if(ncol(imp_values) > 1) {
    imp_scores <- rowMeans(abs(imp_values), na.rm = TRUE)
  } else {
    imp_scores <- imp_values[, 1]
  }
  feature_names <- rownames(imp_values)
} else {
  # If it's a vector
  imp_scores <- as.numeric(imp_values)
  feature_names <- names(imp_values)
}

# Create dataframe
imp_df <- data.frame(
  Feature = feature_names,
  Importance = imp_scores,
  row.names = NULL
)

# Sort by importance
imp_df <- imp_df[order(imp_df$Importance, decreasing = TRUE), ]

# Check if features are PC names or original features
if(grepl("^PC", imp_df$Feature[1])) {
  cat("Features are PC names - back-projecting to original features...\n")

  # Extract PC importance
  n_pcs <- nrow(imp_df)
  pc_imp <- imp_df$Importance[1:n_pcs]
}

```

```

# Get PCA rotation
pca_rot <- svm_linear$preProcess$rotation

# Calculate original feature importance
if(ncol(pca_rot) >= n_pcs) {
  pca_rot_used <- pca_rot[, 1:n_pcs, drop = FALSE]
  feature_imp <- as.vector(abs(pca_rot_used) %*% matrix(pc_imp, ncol = 1))
  feature_names <- rownames(pca_rot_used)

  imp_df <- data.frame(
    Feature = feature_names,
    Importance = feature_imp
  )
  imp_df <- imp_df[order(imp_df$Importance, decreasing = TRUE), ]
}

# Normalize to 0-100 scale for better interpretation
imp_df$Importance_norm <- 100 * imp_df$Importance / max(imp_df$Importance)

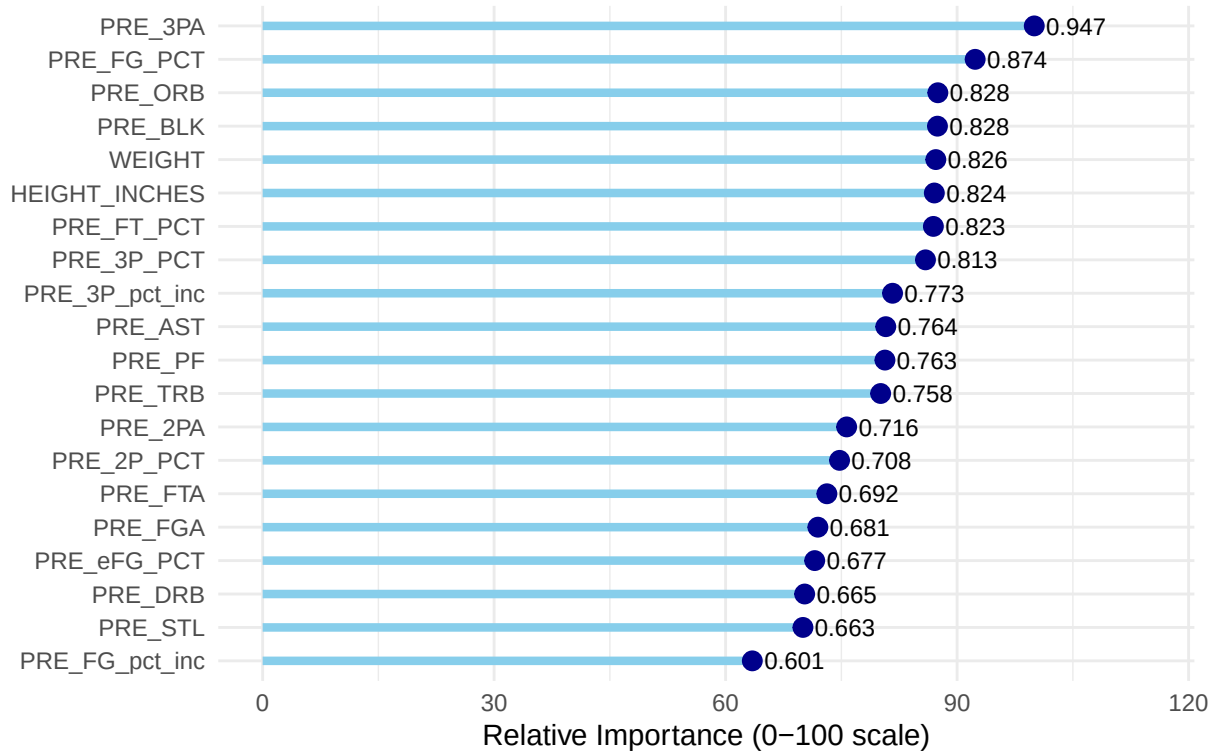
# Top 20 features
top_n <- min(20, nrow(imp_df))
imp_top20 <- imp_df[1:top_n, ]
imp_top20$Feature <- factor(imp_top20$Feature,
                           levels = rev(imp_top20$Feature))

# Plot
ggplot(imp_top20, aes(x = Feature, y = Importance_norm)) +
  geom_segment(aes(xend = Feature, yend = 0),
              color = "skyblue", linewidth = 1.5) +
  geom_point(size = 3, color = "darkblue") +
  geom_text(aes(label = round(Importance, 3)), hjust = -0.2, size = 3) +
  coord_flip() +
  theme_minimal() +
  labs(title = "SVM Linear of Combined Class - Top 20 Most Important Features",
       subtitle = paste0("Cost C = ", svm_linear$bestTune$cost,
                        " | Test Accuracy = ", round(test_acc_linear, 4)),
       x = "",
       y = "Relative Importance (0-100 scale)") +
  expand_limits(y = max(imp_top20$Importance_norm) * 1.15)

```

SVM Linear of Combined Class – Top 20 Most Important Feature

Cost C = 10 | Test Accuracy = 0.5222



```
# Repeat
svm_radial <- train(
  response ~ .,
  data = train_data,
  method = "svmRadial",
  trControl = trainControl(
    method = "cv",
    number = 10,
    classProbs = TRUE,
    summaryFunction = multiClassSummary
  ),
  tuneGrid = expand.grid(
    C = c(0.1, 1, 10),
    sigma = c(0.01, 0.05, 0.1)
  ),
  prob.model = TRUE
)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
print(svm_radial)
```

```
## Support Vector Machines with Radial Basis Function Kernel
##
```

```
## 1186 samples
## 27 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1068, 1066, 1067, 1068, 1069, ...
## Resampling results across tuning parameters:
##
## C      sigma  logLoss  AUC      prAUC      Accuracy  Kappa      Mean_F1
## 0.1  0.01  1.265810  0.7381375  0.4196191  0.4646203  0.3104321  NaN
## 0.1  0.05  1.287806  0.7375152  0.4068036  0.4654448  0.3112115  0.4663253
## 0.1  0.10  1.310718  0.7366377  0.3994815  0.4603675  0.3005422  0.4198721
## 1.0  0.01  1.222721  0.7590793  0.4385806  0.4974802  0.3507443  NaN
## 1.0  0.05  1.232600  0.7623569  0.4287441  0.4924020  0.3434745  0.4387023
## 1.0  0.10  1.266850  0.7505242  0.4112051  0.4788633  0.3241935  NaN
## 10.0 0.01  1.208031  0.7733615  0.4504943  0.5007709  0.3551704  0.4183001
## 10.0 0.05  1.283665  0.7411509  0.4163312  0.4662572  0.3042913  0.4154865
## 10.0 0.10  1.298536  0.7345393  0.4062216  0.4746602  0.3139262  0.4108938
## Mean_Sensitivity Mean_Specificity Mean_Pos_Pred_Value Mean_Neg_Pred_Value
## 0.4281330        0.8616718        0.3433795        0.8661599
## 0.4285888        0.8618326        0.4133188        0.8666540
## 0.4172059        0.8595602        0.3881860        0.8645382
## 0.4538179        0.8697807        0.3881635        0.8737308
## 0.4487395        0.8682413        0.4534697        0.8716830
## 0.4339768        0.8642137        0.3834531        0.8682752
## 0.4574163        0.8707319        0.4601085        0.8739659
## 0.4171408        0.8601026        0.4592905        0.8641455
## 0.4226362        0.8620263        0.4846190        0.8669470
## Mean_Precision Mean_Recall Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.3433795        0.4281330        0.09292406        0.6449024
## 0.4133188        0.4285888        0.09308897        0.6452107
## 0.3881860        0.4172059        0.09207349        0.6383830
## 0.3881635        0.4538179        0.09949604        0.6617993
## 0.4534697        0.4487395        0.09848039        0.6584904
## 0.3834531        0.4339768        0.09577266        0.6490952
## 0.4601085        0.4574163        0.10015418        0.6640741
## 0.4592905        0.4171408        0.09325143        0.6386217
## 0.4846190        0.4226362        0.09493203        0.6423313
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were sigma = 0.01 and C = 10.
```

```
cat("Best parameters: C =", svm_radial$bestTune$C,
    " sigma =", svm_radial$bestTune$sigma, "\n")
```

```
## Best parameters: C = 10 sigma = 0.01
```

```
cat("Cross-validated accuracy (training set):", max(svm_radial$results$Accuracy), "\n")
```

```
## Cross-validated accuracy (training set): 0.5007709
```

```

preds_svm_radial <- predict(svm_radial, newdata = test_data)
preds_svm_radial <- factor(preds_svm_radial, levels = levels(test_data$response))

cm_svm_radial <- confusionMatrix(preds_svm_radial, test_data$response, mode = "prec_recall")
print(cm_svm_radial)

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      50      2      17       8       1
##   MidLow       0      1       0       0       0
##   LowMid      11      3      26      16       2
##   MidHigh       5     15      16      40      16
##   HighHigh      0     10       1      21      32
##
## Overall Statistics
##
##              Accuracy : 0.5085
##              95% CI : (0.4497, 0.5671)
##   No Information Rate : 0.2901
##   P-Value [Acc > NIR] : 3.612e-15
##
##              Kappa : 0.3618
##
## Mcnemar's Test P-Value : 0.0001856
##
## Statistics by Class:
##
##              Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision              0.6410      1.000000      0.44828      0.4348
## Recall                  0.7576      0.032258      0.43333      0.4706
## F1                      0.6944      0.062500      0.44068      0.4520
## Prevalence              0.2253      0.105802      0.20478      0.2901
## Detection Rate          0.1706      0.003413      0.08874      0.1365
## Detection Prevalence    0.2662      0.003413      0.19795      0.3140
## Balanced Accuracy       0.8171      0.516129      0.64800      0.6103
##
##              Class: HighHigh
## Precision              0.5000
## Recall                  0.6275
## F1                      0.5565
## Prevalence              0.1741
## Detection Rate          0.1092
## Detection Prevalence    0.2184
## Balanced Accuracy       0.7476

```

```

test_acc_radial <- mean(as.character(preds_svm_radial) == as.character(test_data$response))
cat("Test Accuracy (SVM Radial):", test_acc_radial, "\n")

```

```

## Test Accuracy (SVM Radial): 0.5085324

```

```

probs_svm_radial <- predict(svm_radial, newdata = test_data, type = "prob")
probs_svm_radial <- as.matrix(probs_svm_radial)

colnames(probs_svm_radial) <- colnames(probs_svm_radial)
probs_svm_radial <- probs_svm_radial[, levels(test_data$response), drop = FALSE]

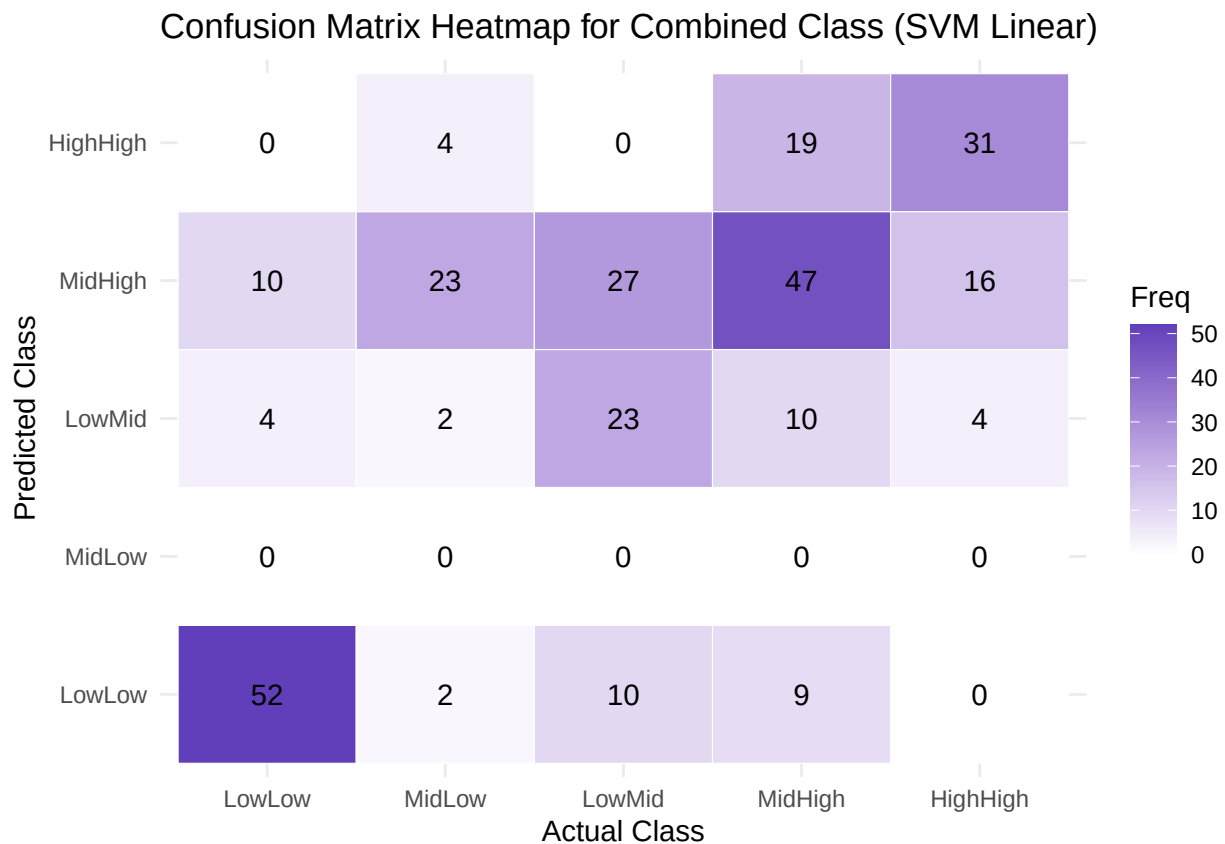
model_results <- rbind(model_results, data.frame(
  Model = "SVM Radial Kernel",
  Accuracy = test_acc_radial,
  Precision = mean(cm_svm_radial$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_svm_radial$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_svm_radial$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_svm_radial)
))

```

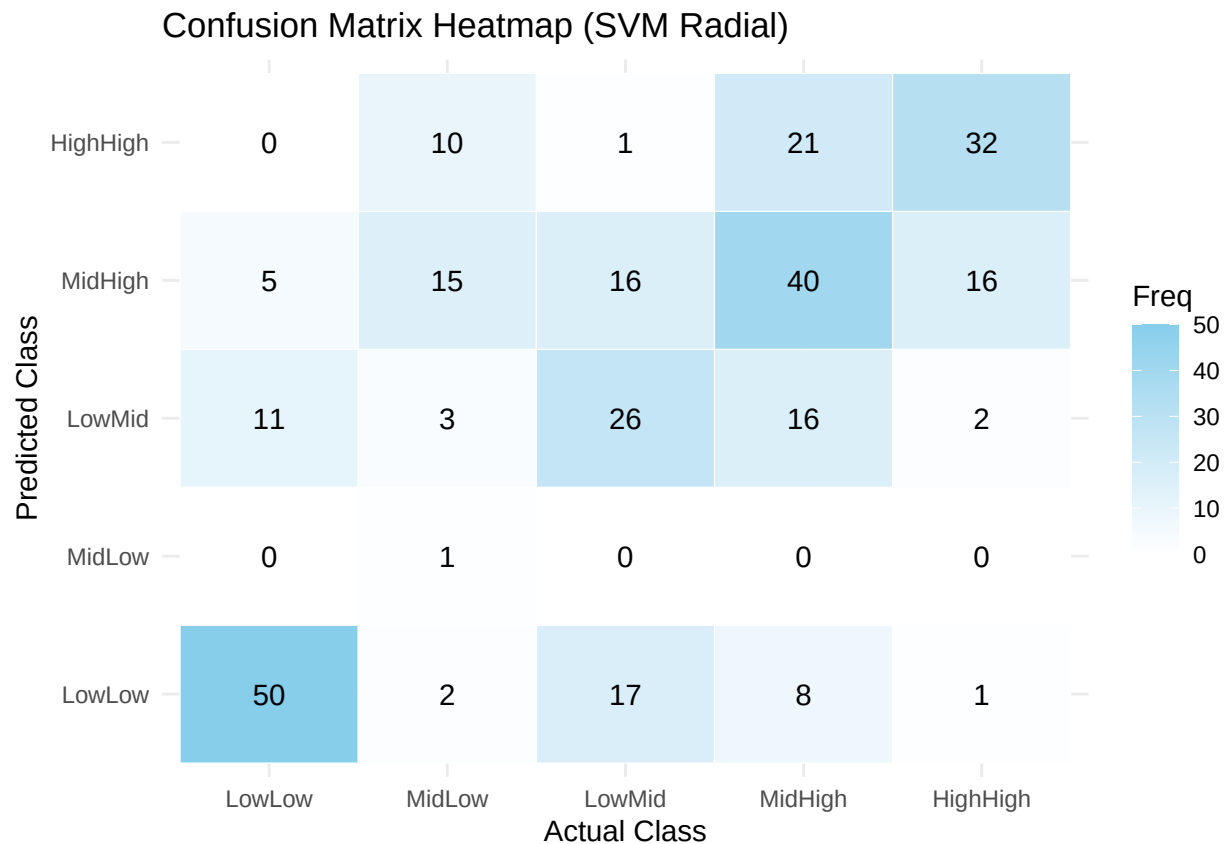
```

cm_table <- as.data.frame(cm_svm_linear$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "#613fbaff") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap for Combined Class (SVM Linear)",
       x = "Actual Class", y = "Predicted Class")

```



```
cm_table <- as.data.frame(cm_svm_radial$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (SVM Radial)",
       x = "Actual Class", y = "Predicted Class")
```



Random Forest

```
set.seed(6767)
# CV for random forest
train_control <- trainControl(method = "cv", number = 10)

tune_grid <- expand.grid(mtry = c(2, 5, 7, 10, 15, 20))

results <- list()
ntree_values <- c(100, 250, 500)

for (nt in ntree_values) {
  rf_model <- train(response ~ ., data = train_data,
                    method = "rf",
```

```

        trControl = train_control,
        tuneGrid = tune_grid,
        ntree = nt)

results[[paste0("ntree_", nt)]] <- rf_model

preds <- predict(rf_model, newdata = test_data)
cm <- confusionMatrix(preds, test_data$response, mode = "prec_recall")

# Print results
cat("\nResults for ntree =", nt, ":\n")
print(rf_model$results)
cat("Best mtry:", rf_model$bestTune$mtry, "\n")
cat("CV Accuracy:", max(rf_model$results$Accuracy), "\n")
}

```

```

##
## Results for ntree = 100 :
##   mtry Accuracy      Kappa AccuracySD      KappaSD
## 1    2 0.5008332 0.3425324 0.02843731 0.03839297
## 2    5 0.5219200 0.3735218 0.01630944 0.02144609
## 3    7 0.5109885 0.3594257 0.03069739 0.03900863
## 4   10 0.5042088 0.3507255 0.01904903 0.02541127
## 5   15 0.5051061 0.3540154 0.03039054 0.04008641
## 6   20 0.5084746 0.3579841 0.03373367 0.04509921
## Best mtry: 5
## CV Accuracy: 0.52192
##
## Results for ntree = 250 :
##   mtry Accuracy      Kappa AccuracySD      KappaSD
## 1    2 0.5195076 0.3653180 0.03084034 0.04098944
## 2    5 0.5260879 0.3779517 0.02865717 0.03988260
## 3    7 0.5093013 0.3554796 0.04112772 0.05559015
## 4   10 0.5100489 0.3587426 0.04014311 0.05433210
## 5   15 0.5151122 0.3651382 0.03645686 0.04837478
## 6   20 0.5084316 0.3566273 0.03134842 0.04258721
## Best mtry: 5
## CV Accuracy: 0.5260879
##
## Results for ntree = 500 :
##   mtry Accuracy      Kappa AccuracySD      KappaSD
## 1    2 0.5033184 0.3447466 0.03382129 0.04658683
## 2    5 0.5151614 0.3626811 0.03877724 0.05294920
## 3    7 0.5100848 0.3565511 0.02817233 0.03797243
## 4   10 0.5092515 0.3571203 0.03749754 0.04985950
## 5   15 0.5076065 0.3548243 0.02844090 0.03737150
## 6   20 0.5059189 0.3543597 0.04449500 0.05827652
## Best mtry: 5
## CV Accuracy: 0.5151614

```

```

# Use best model as fit
rf_model <- train(response ~ ., data = train_data,
                  method = "rf",

```

```

trControl = trainControl(method = "cv", number = 10,
                          summaryFunction = multiClassSummary),
tuneGrid = data.frame(mtry = results[["ntree_500"]]$bestTune$mtry),
ntree = 500)

```

```

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

```

```

preds_rf <- predict(rf_model, newdata = test_data)
cm_rf <- confusionMatrix(preds_rf, test_data$response, mode = "prec_recall")

probs_rf <- predict(rf_model, newdata = test_data, type = "prob")
probs_rf <- as.matrix(probs_rf)

colnames(probs_rf) <- colnames(probs_rf)
probs_rf <- probs_rf[, levels(test_data$response), drop = FALSE]

model_results <- rbind(model_results, data.frame(
  Model = "Base Random Forest",
  Accuracy = mean(preds_rf == test_data$response),
  Precision = mean(cm_rf$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_rf$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_rf$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_rf)
))

print(rf_model)

```

```

## Random Forest
##
## 1186 samples
## 27 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1068, 1067, 1068, 1067, 1067, 1066, ...
## Resampling results:
##
## Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 0.5201402 0.3681794 NaN 0.4527455 0.8726608
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.440002 0.8777627 0.440002 0.4527455
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.104028 0.6627032
##
## Tuning parameter 'mtry' was held constant at a value of 5

```

```

print(cm_rf)

```

```

## Confusion Matrix and Statistics
##

```

```

##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      46      2      11       8       1
##   MidLow       0      0       0       0       0
##   LowMid      12      2      20      10       1
##   MidHigh       7     21      29      50      23
##   HighHigh      1      6       0      17      26
##
## Overall Statistics
##
##           Accuracy : 0.4846
##           95% CI : (0.4261, 0.5435)
##   No Information Rate : 0.2901
##   P-Value [Acc > NIR] : 1.841e-12
##
##           Kappa : 0.32
##
## McNemar's Test P-Value : 6.722e-06
##
## Statistics by Class:
##
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision           0.6765           NA           0.44444           0.3846
## Recall              0.6970           0.0000           0.33333           0.5882
## F1                  0.6866           NA           0.38095           0.4651
## Prevalence          0.2253           0.1058           0.20478           0.2901
## Detection Rate      0.1570           0.0000           0.06826           0.1706
## Detection Prevalence 0.2321           0.0000           0.15358           0.4437
## Balanced Accuracy    0.8000           0.5000           0.61302           0.6018
##
##           Class: HighHigh
## Precision           0.52000
## Recall              0.50980
## F1                  0.51485
## Prevalence          0.17406
## Detection Rate      0.08874
## Detection Prevalence 0.17065
## Balanced Accuracy    0.70532

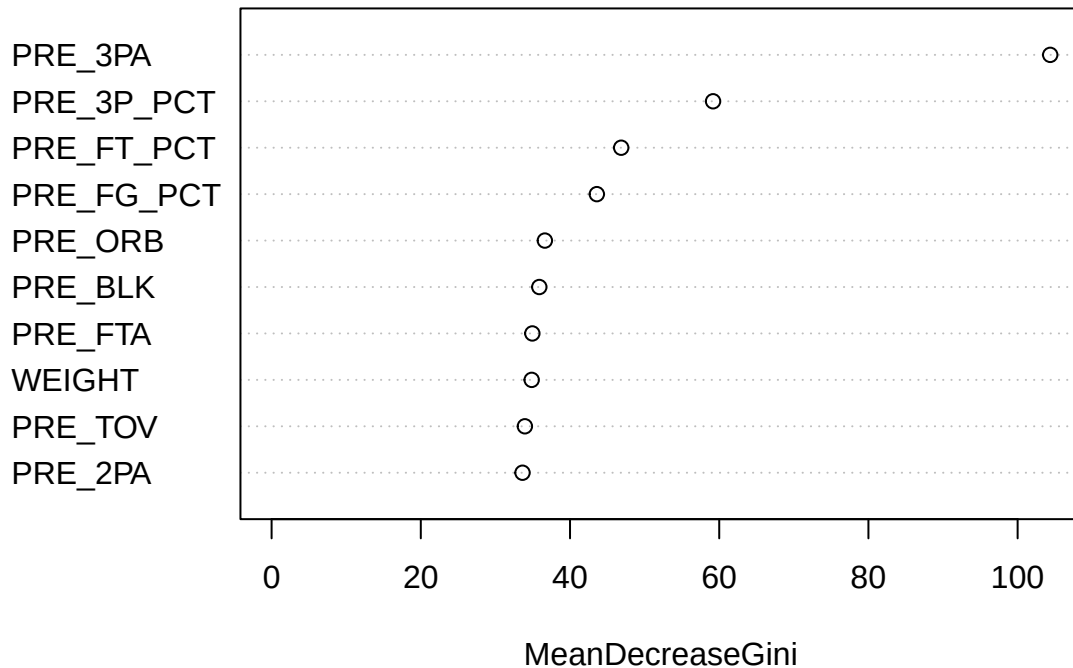
```

```

# Print Variable Importance Plot
varImpPlot(rf_model$finalModel, n.var = 10, main = "Variable Importance for Best Random Forest Model")

```

Variable Importance for Best Random Forest Model



```
cm_table <- as.data.frame(cm_rf$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "skyblue") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap for Random Forest of Combined Class",
       x = "Actual Class", y = "Predicted Class")
```

Confusion Matrix Heatmap for Random Forest of Combined Class



Upsampling

```
set.seed(67)

# Cross validation for upsampling
train_control <- trainControl(method = "cv", number = 10,
                              sampling = "up",
                              summaryFunction = multiClassSummary)

rf_model_balanced <- train(response ~ ., data = train_data,
                           method = "rf",
                           trControl = train_control,
                           tuneGrid = data.frame(mtry = c(2, 5, 10, 12, 15, 20)),
                           ntree = 500)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

print(rf_model_balanced)

## Random Forest
##
## 1186 samples
```

```
## 27 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1067, 1067, 1068, 1068, 1067, 1066, ...
## Additional sampling using up-sampling
##
## Resampling results across tuning parameters:
##
## mtry Accuracy Kappa Mean_F1 Mean_Sensitivity Mean_Specificity
## 2 0.5066280 0.3579724 0.4626606 0.4558197 0.8709739
## 5 0.5175538 0.3735908 0.4814094 0.4674086 0.8743334
## 10 0.5091861 0.3652468 0.4794498 0.4674164 0.8726567
## 12 0.5075767 0.3643431 0.4653746 0.4637465 0.8728001
## 15 0.5058824 0.3625377 0.4623991 0.4626567 0.8725236
## 20 0.5042363 0.3607163 0.4618516 0.4623397 0.8721591
## Mean_Pos_Pred_Value Mean_Neg_Pred_Value Mean_Precision Mean_Recall
## 0.4575902 0.8739556 0.4575902 0.4558197
## 0.4656626 0.8767613 0.4656626 0.4674086
## 0.4661615 0.8743503 0.4661615 0.4674164
## 0.4610279 0.8743401 0.4610279 0.4637465
## 0.4596569 0.8741495 0.4596569 0.4626567
## 0.4660612 0.8737114 0.4660612 0.4623397
## Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.1013256 0.6633968
## 0.1035108 0.6708710
## 0.1018372 0.6700366
## 0.1015153 0.6682733
## 0.1011765 0.6675901
## 0.1008473 0.6672494
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 5.
```

```
cat("Best mtry:", rf_model_balanced$bestTune$mtry, "\n")
```

```
## Best mtry: 5
```

```
cat("Cross-validated accuracy:", max(rf_model_balanced$results$Accuracy), "\n")
```

```
## Cross-validated accuracy: 0.5175538
```

```
preds_rf_up <- predict(rf_model_balanced, newdata = test_data)
cm_rf_up <- confusionMatrix(preds_rf_up, test_data$response, mode = "prec_recall")

probs_rf_up <- predict(rf_model_balanced, newdata = test_data, type = "prob")
probs_rf_up <- as.matrix(probs_rf_up)

colnames(probs_rf_up) <- colnames(probs_rf_up)
probs_rf_up <- probs_rf_up[, levels(test_data$response), drop = FALSE]
```

```

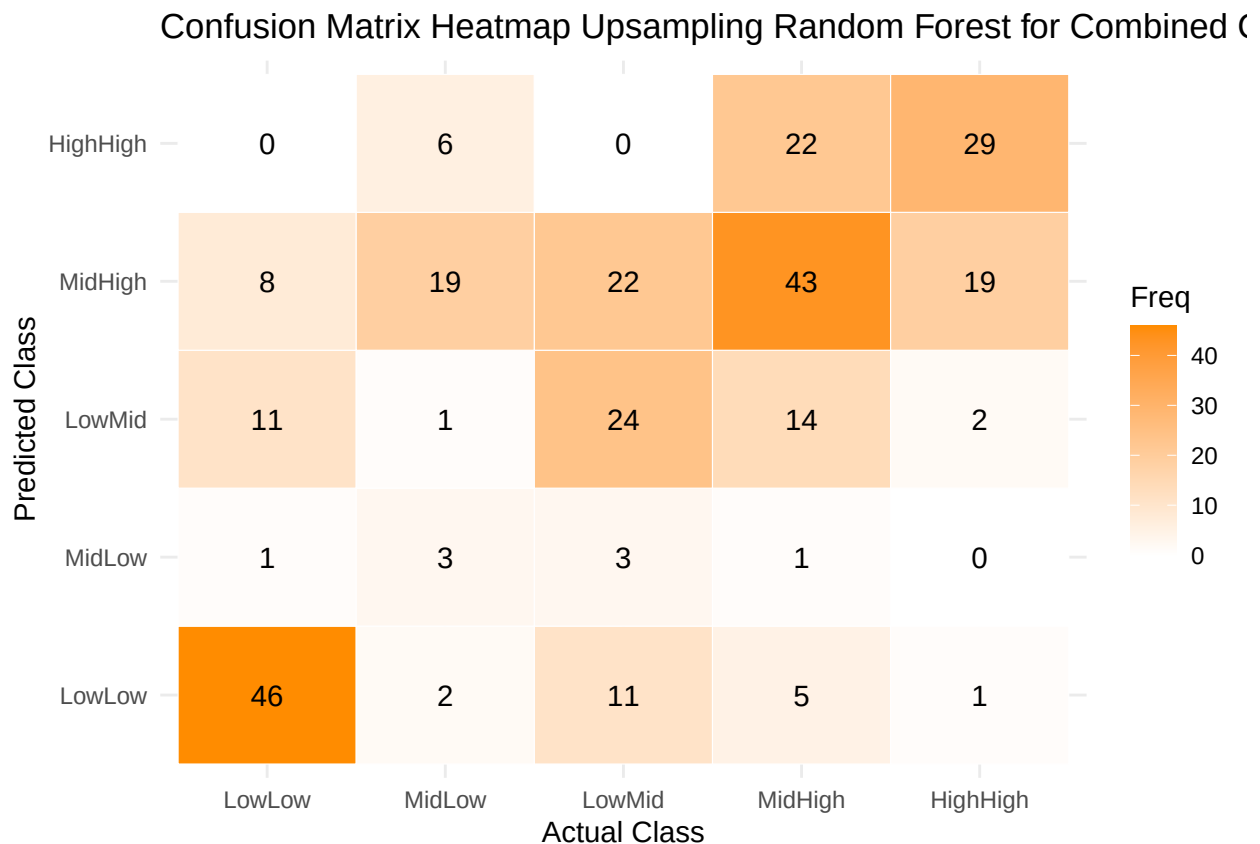
model_results <- rbind(model_results, data.frame(
  Model = "Upsampling Random Forest",
  Accuracy = mean(preds_rf_up == test_data$response),
  Precision = mean(cm_rf_up$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_rf_up$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_rf_up$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_rf_up)
))

```

```

cm_table <- as.data.frame(cm_rf_up$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap Upsampling Random Forest for Combined Class",
       x = "Actual Class", y = "Predicted Class")

```



Class Weights

```

set.seed(12)
# Set class weights
class_counts <- table(train_data$response)

```

```

class_weights <- 1 / class_counts
class_weights <- class_weights / sum(class_weights)

# CV for class weights
rf_weighted <- randomForest(response ~ ., data = train_data,
                             ntree = 500,
                             mtry = 7,
                             importance = TRUE,
                             classwt = class_weights)

preds_rf_weighted <- predict(rf_weighted, newdata = test_data)

cm_rf_weighted <- confusionMatrix(preds_rf_weighted, test_data$response, mode = "prec_recall")
accuracy_rf_weighted <- mean(as.character(preds_rf_weighted) == as.character(test_data$response))
cat("Test Accuracy:", accuracy_rf_weighted, "\n")

```

```
## Test Accuracy: 0.4709898
```

```
print(cm_rf_weighted)
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      44      1      10       6       2
##   MidLow       0      1       4       2       0
##   LowMid      14      2      20      11       1
##   MidHigh       8     19      25      50      25
##   HighHigh      0      8       1      16      23
##
```

```
## Overall Statistics
```

```
##
##           Accuracy : 0.471
##           95% CI : (0.4127, 0.5299)
##   No Information Rate : 0.2901
##   P-Value [Acc > NIR] : 4.831e-11
##
```

```
##           Kappa : 0.3051
##
```

```
## McNemar's Test P-Value : 0.0001996
##
```

```
## Statistics by Class:
```

```
##
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Precision           0.6984           0.142857           0.41667           0.3937
## Recall              0.6667           0.032258           0.33333           0.5882
## F1                  0.6822           0.052632           0.37037           0.4717
## Prevalence          0.2253           0.105802           0.20478           0.2901
## Detection Rate      0.1502           0.003413           0.06826           0.1706
## Detection Prevalence 0.2150           0.023891           0.16382           0.4334
## Balanced Accuracy    0.7915           0.504679           0.60658           0.6090
##           Class: HighHigh
```

```
## Precision          0.4792
## Recall             0.4510
## F1                 0.4646
## Prevalence         0.1741
## Detection Rate     0.0785
## Detection Prevalence 0.1638
## Balanced Accuracy  0.6738
```

```
probs_rf_weighted <- predict(rf_weighted, newdata = test_data, type = "prob")
probs_rf_weighted <- as.matrix(probs_rf_weighted)

colnames(probs_rf_weighted) <- colnames(probs_rf_weighted)
probs_rf_weighted <- probs_rf_weighted[, levels(test_data$response), drop = FALSE]

model_results <- rbind(model_results, data.frame(
  Model = "Weighted Random Forest Non-CV",
  Accuracy = accuracy_rf_weighted,
  Precision = mean(cm_rf_weighted$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_rf_weighted$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_rf_weighted$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_rf_weighted)
))
```

```
cm_table <- as.data.frame(cm_rf_weighted$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "forestgreen") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap Weighted Random Forest For Combined Class",
       x = "Actual Class", y = "Predicted Class")
```

Confusion Matrix Heatmap Weighted Random Forest For Combined Cl



```
set.seed(12)

# Set class weights
class_counts <- table(train_data$response)
class_weights <- 1 / class_counts
class_weights <- class_weights / sum(class_weights)

# Calculate observation-level weights
obs_weights <- class_weights[as.character(train_data$response)]

# 10-fold CV with caret - Fixed version
rf_weighted_cv <- train(
  response ~ .,
  data = train_data,
  method = "rf",
  trControl = trainControl(
    method = "cv",
    number = 10,
    classProbs = TRUE,
    summaryFunction = multiClassSummary,
    verboseIter = FALSE
  ),
  ntree = 500,
  # Use tuneGrid instead of mtry parameter
  tuneGrid = data.frame(mtry = 7), # Fixed mtry value
  importance = TRUE,
```

```

weights = obs_weights
)

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

# CV results
cat("\n===== 10-Fold Cross-Validation Results =====\n")

##
## ===== 10-Fold Cross-Validation Results =====

print(rf_weighted_cv)

## Random Forest
##
## 1186 samples
## 27 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1068, 1068, 1068, 1068, 1067, 1067, ...
## Resampling results:
##
##   logLoss   AUC      prAUC      Accuracy   Kappa      Mean_F1
## 1.189316 0.7768618 0.4512925 0.5260718 0.3781813 NaN
## Mean_Sensitivity Mean_Specificity Mean_Pos_Pred_Value Mean_Neg_Pred_Value
## 0.4604364      0.8748493      0.4511592      0.8796099
## Mean_Precision Mean_Recall Mean_Detection_Rate Mean_Balanced_Accuracy
## 0.4511592      0.4604364      0.1052144      0.6676429
##
## Tuning parameter 'mtry' was held constant at a value of 7

cat("Cross-validated Accuracy:", max(rf_weighted_cv$results$Accuracy), "\n")

## Cross-validated Accuracy: 0.5260718

# Test set predictions
preds_rf_weighted <- predict(rf_weighted_cv, newdata = test_data)

cm_rf_weighted <- confusionMatrix(preds_rf_weighted, test_data$response, mode = "prec_recall")
accuracy_rf_weighted <- mean(as.character(preds_rf_weighted) == as.character(test_data$response))

cat("\n===== Test Set Results =====\n")

##
## ===== Test Set Results =====

```

```
cat("Test Accuracy:", accuracy_rf_weighted, "\n")
```

```
## Test Accuracy: 0.4914676
```

```
print(cm_rf_weighted)
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction LowLow MidLow LowMid MidHigh HighHigh
```

```
##   LowLow      46      1     11      7      1
```

```
##   MidLow       0      0      0      0      0
```

```
##   LowMid      13      2     20      9      1
```

```
##   MidHigh      7     20     29     52     23
```

```
##   HighHigh     0      8      0     17     26
```

```
##
```

```
## Overall Statistics
```

```
##
```

```
##           Accuracy : 0.4915
```

```
##           95% CI : (0.4329, 0.5503)
```

```
##   No Information Rate : 0.2901
```

```
##   P-Value [Acc > NIR] : 3.316e-13
```

```
##
```

```
##           Kappa : 0.3289
```

```
##
```

```
##   McNemar's Test P-Value : 2.575e-06
```

```
##
```

```
## Statistics by Class:
```

```
##
```

```
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
```

```
## Precision           0.6970           NA           0.44444           0.3969
```

```
## Recall              0.6970           0.0000           0.33333           0.6118
```

```
## F1                  0.6970           NA           0.38095           0.4815
```

```
## Prevalence          0.2253           0.1058           0.20478           0.2901
```

```
## Detection Rate      0.1570           0.0000           0.06826           0.1775
```

```
## Detection Prevalence 0.2253           0.0000           0.15358           0.4471
```

```
## Balanced Accuracy    0.8044           0.5000           0.61302           0.6160
```

```
##           Class: HighHigh
```

```
## Precision           0.50980
```

```
## Recall              0.50980
```

```
## F1                  0.50980
```

```
## Prevalence          0.17406
```

```
## Detection Rate      0.08874
```

```
## Detection Prevalence 0.17406
```

```
## Balanced Accuracy    0.70325
```

```
probs_rf_weighted <- predict(rf_weighted_cv, newdata = test_data, type = "prob")
```

```
probs_rf_weighted <- as.matrix(probs_rf_weighted)
```

```
colnames(probs_rf_weighted) <- colnames(probs_rf_weighted)
```

```
probs_rf_weighted <- probs_rf_weighted[, levels(test_data$response), drop = FALSE]
```

```

model_results <- rbind(model_results, data.frame(
  Model = "Weighted Random Forest CV",
  Accuracy = accuracy_rf_weighted,
  Precision = mean(cm_rf_weighted$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_rf_weighted$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_rf_weighted$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_rf_weighted)
))

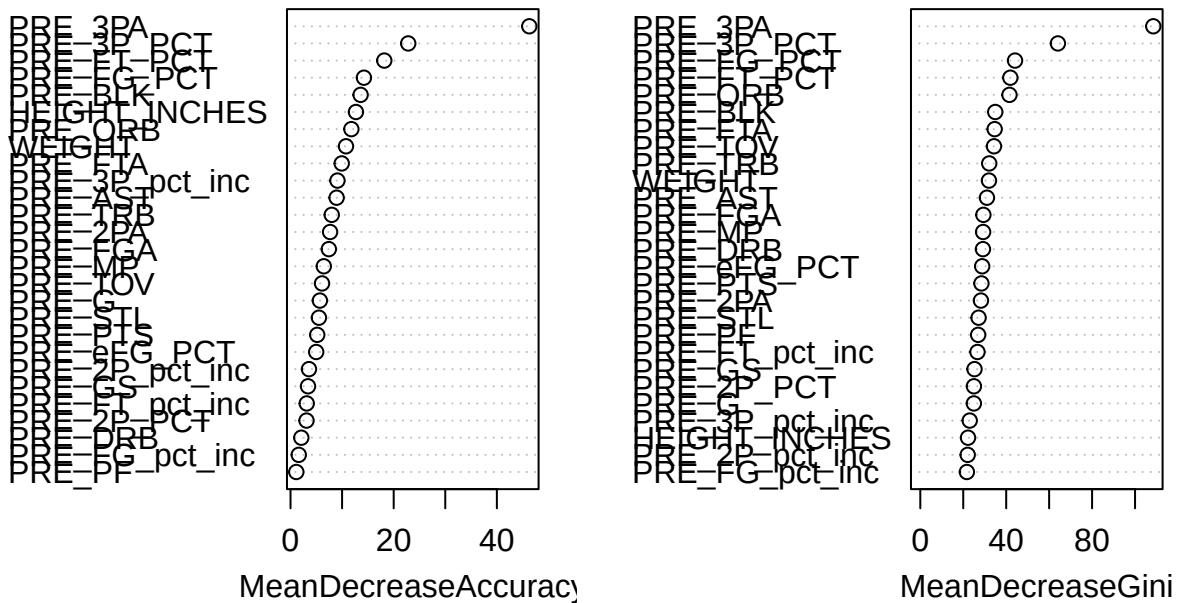
```

```

# Original best model, so we check the VarImpPlot
varImpPlot(rf_weighted, main="RF Weighted Variable Importance")

```

RF Weighted Variable Importance



```

# Original best model, so we print a pretty tree
single_tree <- randomForest::getTree(rf_weighted, k = 1, labelVar = TRUE)

to_rpart <- function(tree, data) {
  response <- all.vars(formula(rf_weighted))[1]
  form <- as.formula(paste(response, "~ ."))
  rpart(form, data = data, control = rpart.control(cp = 0.0001))
}

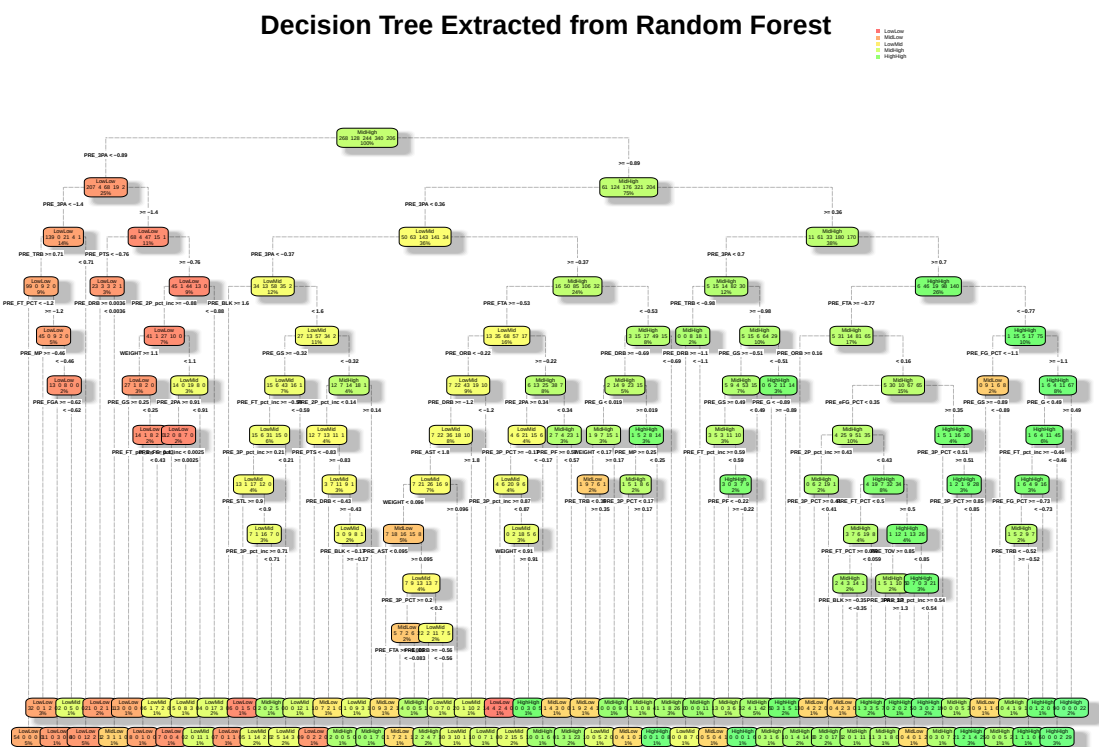
rpart_tree <- to_rpart(single_tree, train_data)
rpart.plot(
  rpart_tree,
  type = 4,

```

```
extra = 101,  
fallen.leaves = TRUE,  
box.palette = "RdYlGn",  
branch.lty = 3,  
shadow.col = "gray",  
main = "Decision Tree Extracted from Random Forest"  
)
```

Warning: Cannot retrieve the data used to build the model (model.frame: variable lengths differ (four
To silence this warning:
Call rpart.plot with roundint=FALSE,
or rebuild the rpart model with model=TRUE.

Warning: labs do not fit even at cex 0.15, there may be some overplotting



XGBoost

```
set.seed(9999)  
  
# Cross Validation for XGBoost  
train_control <- trainControl(method = "cv", number = 10)  
tune_grid <- expand.grid(
```

```

nrounds = c(100, 250, 500),
max_depth = c(3, 5, 7),
eta = c(0.01, 0.1, 0.3),
gamma = 0,
colsample_bytree = 0.8,
min_child_weight = 1,
subsample = 0.8
)

xgb_model <- train(
  response ~ ., data = train_data,
  method = "xgbTree",
  trControl = train_control,
  tuneGrid = tune_grid,
  verbose = FALSE
)

```

```

## [10:42:43] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:42:43] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:42:55] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:42:55] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:09] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:09] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:15] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:15] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:25] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:25] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:35] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:35] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:41] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:41] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:47] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:47] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:55] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:43:55] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:44:00] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:44:00] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:44:12] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:44:12] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:44:30] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:44:30] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:44:35] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:44:35] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:44:45] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:44:45] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:44:56] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:44:56] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:45:01] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:45:01] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:45:08] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:45:08] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:45:15] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead

```

[illegible]


```

## [10:52:26] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:52:32] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:52:32] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:52:44] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:52:44] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:02] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:02] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:08] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:08] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:18] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:18] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:29] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:29] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:34] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:34] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:41] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:41] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:48] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:48] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:54] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:53:54] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:54:05] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:54:05] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:54:23] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:54:23] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:54:28] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:54:28] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:54:38] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:54:38] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:54:49] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:54:49] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:54:54] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:54:54] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:55:01] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:55:01] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:55:08] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead
## [10:55:08] WARNING: src/c_api/c_api.cc:935: 'ntree_limit' is deprecated, use 'iteration_range' instead

```

```
print(xgb_model)
```

```

## eXtreme Gradient Boosting
##
## 1186 samples
## 27 predictor
## 5 classes: 'LowLow', 'MidLow', 'LowMid', 'MidHigh', 'HighHigh'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1069, 1066, 1069, 1069, 1067, 1066, ...
## Resampling results across tuning parameters:
##
##  eta    max_depth  nrounds  Accuracy    Kappa
##  0.01    3          100      0.5032973  0.3444251
##  0.01    3          250      0.4922859  0.3316637

```

```
## 0.01 3 500 0.4998424 0.3443383
## 0.01 5 100 0.5083041 0.3529285
## 0.01 5 250 0.4981335 0.3417283
## 0.01 5 500 0.4922509 0.3359405
## 0.01 7 100 0.4990750 0.3436164
## 0.01 7 250 0.4990605 0.3446290
## 0.01 7 500 0.4931062 0.3374402
## 0.10 3 100 0.4922236 0.3376971
## 0.10 3 250 0.4845593 0.3303759
## 0.10 3 500 0.4754296 0.3210519
## 0.10 5 100 0.4855992 0.3292741
## 0.10 5 250 0.4822311 0.3274073
## 0.10 5 500 0.4704292 0.3144016
## 0.10 7 100 0.4863758 0.3310957
## 0.10 7 250 0.4796384 0.3241383
## 0.10 7 500 0.4787196 0.3254445
## 0.30 3 100 0.4762979 0.3215116
## 0.30 3 250 0.4637418 0.3053172
## 0.30 3 500 0.4569690 0.2984239
## 0.30 5 100 0.4586430 0.2983826
## 0.30 5 250 0.4595188 0.3013384
## 0.30 5 500 0.4595190 0.3022702
## 0.30 7 100 0.4863684 0.3324019
## 0.30 7 250 0.4838683 0.3316960
## 0.30 7 500 0.4829710 0.3309322
##
## Tuning parameter 'gamma' was held constant at a value of 0
## Tuning
##
## Tuning parameter 'min_child_weight' was held constant at a value of 1
##
## Tuning parameter 'subsample' was held constant at a value of 0.8
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were nrounds = 100, max_depth = 5, eta
## = 0.01, gamma = 0, colsample_bytree = 0.8, min_child_weight = 1 and
## subsample = 0.8.
```

```
cat("Best parameters:\n")
```

```
## Best parameters:
```

```
print(xgb_model$bestTune)
```

```
## nrounds max_depth eta gamma colsample_bytree min_child_weight subsample
## 4 100 5 0.01 0 0.8 1 0.8
```

```
cat("Cross-validated accuracy:", max(xgb_model$results$Accuracy), "\n")
```

```
## Cross-validated accuracy: 0.5083041
```

```

preds_xgb <- predict(xgb_model, newdata = test_data)
preds_xgb <- factor(preds_xgb, levels = levels(test_data$response))

cm_xgb <- confusionMatrix(preds_xgb, test_data$response)
print(cm_xgb)

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      48      3      12       7       2
##   MidLow       0      0       1       0       0
##   LowMid      10      0      18       8       0
##   MidHigh       8     23      29      53      25
##   HighHigh      0      5       0      17      24
##
## Overall Statistics
##
##           Accuracy : 0.4881
##           95% CI : (0.4295, 0.5469)
##   No Information Rate : 0.2901
##   P-Value [Acc > NIR] : 7.865e-13
##
##           Kappa : 0.3221
##
##   McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
## Sensitivity           0.7273           0.000000           0.30000           0.6235
## Specificity           0.8943           0.996183           0.92275           0.5913
## Pos Pred Value        0.6667           0.000000           0.50000           0.3841
## Neg Pred Value        0.9186           0.893836           0.83658           0.7935
## Prevalence            0.2253           0.105802           0.20478           0.2901
## Detection Rate        0.1638           0.000000           0.06143           0.1809
## Detection Prevalence  0.2457           0.003413           0.12287           0.4710
## Balanced Accuracy      0.8108           0.498092           0.61137           0.6074
##
##           Class: HighHigh
## Sensitivity           0.47059
## Specificity           0.90909
## Pos Pred Value        0.52174
## Neg Pred Value        0.89069
## Prevalence            0.17406
## Detection Rate        0.08191
## Detection Prevalence  0.15700
## Balanced Accuracy      0.68984

```

```

set.seed(9999)
# Another cross validation
trainControl(method = "cv", number = 10, summaryFunction = multiClassSummary)

```

```
## $method
```

```

## [1] "cv"
##
## $number
## [1] 10
##
## $repeats
## [1] NA
##
## $search
## [1] "grid"
##
## $p
## [1] 0.75
##
## $initialWindow
## NULL
##
## $horizon
## [1] 1
##
## $fixedWindow
## [1] TRUE
##
## $skip
## [1] 0
##
## $verboseIter
## [1] FALSE
##
## $returnData
## [1] TRUE
##
## $returnResamp
## [1] "final"
##
## $savePredictions
## [1] FALSE
##
## $classProbs
## [1] FALSE
##
## $summaryFunction
## function (data, lev = NULL, model = NULL)
## {
##     if (!all(levels(data[, "pred"]) == levels(data[, "obs"])))
##         stop("levels of observed and predicted data do not match")
##     has_class_probs <- all(lev %in% colnames(data))
##     if (has_class_probs) {
##         lloss <- mnLogLoss(data = data, lev = lev, model = model)
##         requireNamespaceQuietStop("pROC")
##         requireNamespaceQuietStop("MLmetrics")
##         prob_stats <- lapply(levels(data[, "pred"]), function(x) {
##             obs <- ifelse(data[, "obs"] == x, 1, 0)
##             prob <- data[, x]

```

```

##         roc_auc <- try(pROC::roc(obs, data[, x], direction = "<",
##             quiet = TRUE), silent = TRUE)
##         roc_auc <- if (inherits(roc_auc, "try-error"))
##             NA
##         else roc_auc$auc
##         pr_auc <- try(MLmetrics::PRAUC(y_pred = data[, x],
##             y_true = obs), silent = TRUE)
##         if (inherits(pr_auc, "try-error"))
##             pr_auc <- NA
##         res <- c(ROC = roc_auc, AUC = pr_auc)
##         return(res)
##     })
##     prob_stats <- do.call("rbind", prob_stats)
##     prob_stats <- colMeans(prob_stats, na.rm = TRUE)
## }
## CM <- confusionMatrix(data[, "pred"], data[, "obs"], mode = "everything")
## if (length(levels(data[, "pred"])) == 2) {
##     class_stats <- CM$byClass
## }
## else {
##     class_stats <- colMeans(CM$byClass)
##     names(class_stats) <- paste("Mean", names(class_stats))
## }
## overall_stats <- if (has_class_probs)
##     c(CM$overall, logLoss = as.numeric(lloss), AUC = unname(prob_stats["ROC"]),
##     prAUC = unname(prob_stats["AUC"]))
## else CM$overall
## stats <- c(overall_stats, class_stats)
## stats <- stats[!names(stats) %in% c("AccuracyNull", "AccuracyLower",
##     "AccuracyUpper", "AccuracyPValue", "McnemarPValue", "Mean Prevalence",
##     "Mean Detection Prevalence")]
## names(stats) <- gsub("[:blank:]", "_", names(stats))
## stat_list <- c("Accuracy", "Kappa", "Mean_F1", "Mean_Sensitivity",
##     "Mean_Specificity", "Mean_Pos_Pred_Value", "Mean_Neg_Pred_Value",
##     "Mean_Precision", "Mean_Recall", "Mean_Detection_Rate",
##     "Mean_Balanced_Accuracy")
## if (has_class_probs)
##     stat_list <- c("logLoss", "AUC", "prAUC", stat_list)
## if (length(levels(data[, "pred"])) == 2)
##     stat_list <- gsub("^Mean_", "", stat_list)
## stats <- stats[c(stat_list)]
## return(stats)
## }
## <bytecode: 0x0000016d764562a8>
## <environment: namespace:caret>
##
## $selectionFunction
## [1] "best"
##
## $preProcOptions
## $preProcOptions$thresh
## [1] 0.95
##
## $preProcOptions$ICAcomp

```

```

## [1] 3
##
## $preProcOptions$k
## [1] 5
##
## $preProcOptions$freqCut
## [1] 19
##
## $preProcOptions$uniqueCut
## [1] 10
##
## $preProcOptions$cutoff
## [1] 0.9
##
##
## $sampling
## NULL
##
## $index
## NULL
##
## $indexOut
## NULL
##
## $indexFinal
## NULL
##
## $timingSamps
## [1] 0
##
## $predictionBounds
## [1] FALSE FALSE
##
## $seeds
## [1] NA
##
## $adaptive
## $adaptive$min
## [1] 5
##
## $adaptive$alpha
## [1] 0.05
##
## $adaptive$method
## [1] "gls"
##
## $adaptive$complete
## [1] TRUE
##
##
## $trim
## [1] FALSE
##
## $allowParallel

```

```
## [1] TRUE
```

```
tune_grid <- expand.grid(
  nrounds = 100,
  max_depth = 5,
  eta = 0.01,
  gamma = 0,
  colsample_bytree = 0.8,
  min_child_weight = 1,
  subsample = 0.8
)

xgb_model <- train(response ~ ., data = train_data,
  method = "xgbTree",
  trControl = trainControl(method = "cv", number = 10),
  tuneGrid = tune_grid,
  verbose = FALSE)

cat("Cross-validated accuracy:", max(xgb_model$results$Accuracy), "\n")
```

```
## Cross-validated accuracy: 0.4965245
```

```
preds_xgb <- predict(xgb_model, newdata = test_data)
preds_xgb <- factor(preds_xgb, levels = levels(test_data$response))

cm_xgb <- confusionMatrix(preds_xgb, test_data$response, mode = "prec_recall")
print(cm_xgb)
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction LowLow MidLow LowMid MidHigh HighHigh
##   LowLow      49      2      12       5       2
##   MidLow       0      0       1       0       0
##   LowMid       8      1      18       7       0
##   MidHigh      9     23      29      53      27
##   HighHigh     0      5       0      20      22
```

```
##
```

```
## Overall Statistics
```

```
##
##           Accuracy : 0.4846
##           95% CI : (0.4261, 0.5435)
##   No Information Rate : 0.2901
##   P-Value [Acc > NIR] : 1.841e-12
```

```
##
```

```
##           Kappa : 0.317
```

```
##
```

```
##   McNemar's Test P-Value : NA
```

```
##
```

```
## Statistics by Class:
```

```
##
```

```
##           Class: LowLow Class: MidLow Class: LowMid Class: MidHigh
```

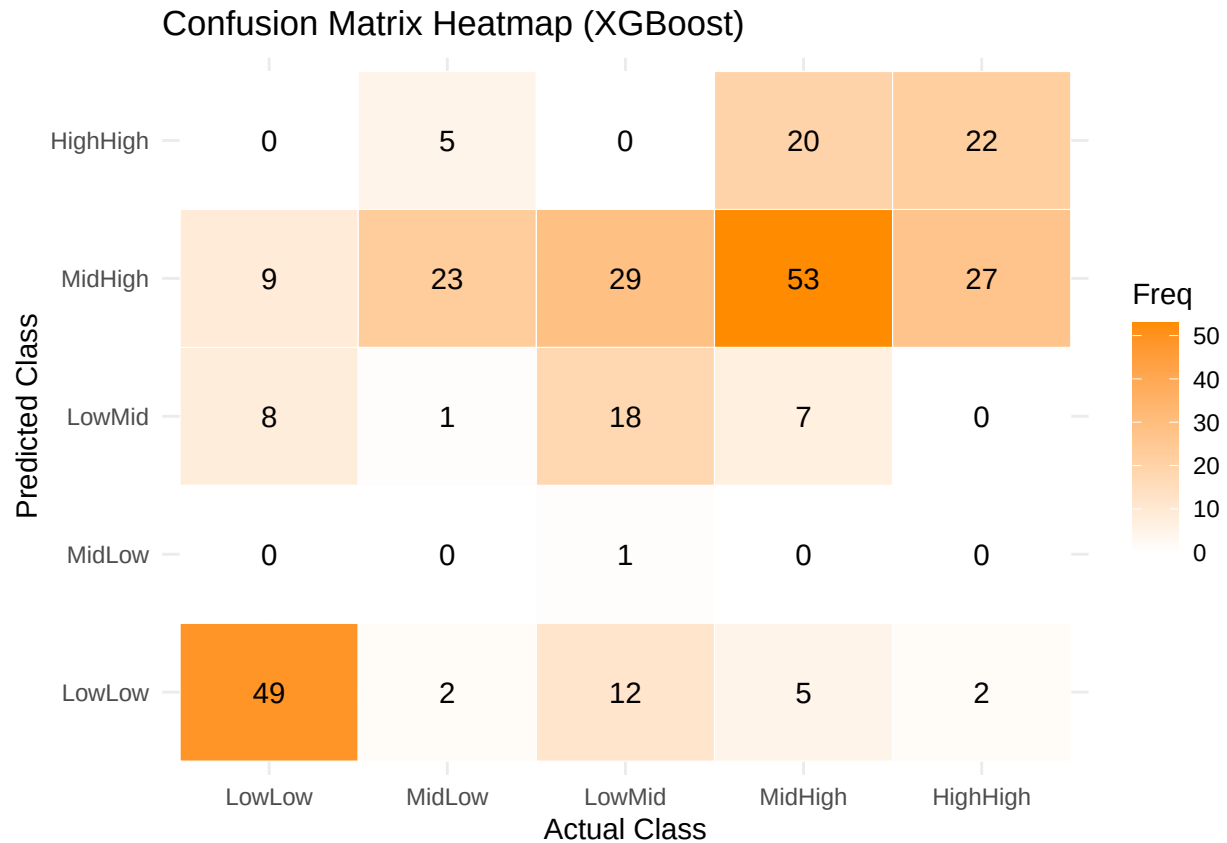
```
## Precision          0.7000      0.000000      0.52941      0.3759
## Recall             0.7424      0.000000      0.30000      0.6235
## F1                 0.7206      NaN          0.38298      0.4690
## Prevalence         0.2253      0.105802      0.20478      0.2901
## Detection Rate     0.1672      0.000000      0.06143      0.1809
## Detection Prevalence 0.2389      0.003413      0.11604      0.4812
## Balanced Accuracy  0.8250      0.498092      0.61567      0.6002
##
## Class: HighHigh
## Precision          0.46809
## Recall             0.43137
## F1                 0.44898
## Prevalence         0.17406
## Detection Rate     0.07509
## Detection Prevalence 0.16041
## Balanced Accuracy  0.66403
```

```
probs_xgb <- predict(xgb_model, newdata = test_data, type = "prob")
probs_xgb <- as.matrix(probs_xgb)

colnames(probs_xgb) <- colnames(probs_xgb)
probs_xgb <- probs_xgb[, levels(test_data$response), drop = FALSE]

model_results <- rbind(model_results, data.frame(
  Model = "XGBoost",
  Accuracy = max(xgb_model$results$Accuracy),
  Precision = mean(cm_xgb$byClass[, "Precision"], na.rm = TRUE),
  Recall = mean(cm_xgb$byClass[, "Recall"], na.rm = TRUE),
  F1 = mean(cm_xgb$byClass[, "F1"], na.rm = TRUE),
  AUC = macro_auc(test_data$response, probs_xgb)
))
```

```
cm_table <- as.data.frame(cm_xgb$table)
ggplot(cm_table, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), color = "black", size = 4) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(title = "Confusion Matrix Heatmap (XGBoost)",
       x = "Actual Class", y = "Predicted Class")
```



Model Results

```
print(model_results)
```

```
##           Model  Accuracy Precision   Recall      F1
## 1           LDA  0.4607509 0.4776979 0.4135294 0.4934686
## 2           QDA  0.4744027 0.4357303 0.4197849 0.4001403
## 3      LASSO Min Lambda 0.4778157 0.4885430 0.4201604 0.4912253
## 4      LASSO 1se Lambda 0.4812287 0.5031451 0.4153298 0.4844480
## 5           PCA  0.4539249 0.4730732 0.4027273 0.4783739
## 6      SVM Linear 0.5221843 0.5508501 0.4663993 0.5593007
## 7  SVM Radial Kernel 0.5085324 0.6048168 0.4642413 0.4412243
## 8   Base Random Forest 0.4846416 0.5063826 0.4256684 0.5118718
## 9  Upsampling Random Forest 0.4948805 0.4880780 0.4536507 0.4521040
## 10 Weighted Random Forest Non-CV 0.4709898 0.4261608 0.4142948 0.4083034
## 11 Weighted Random Forest CV 0.4914676 0.5120412 0.4303743 0.5173019
## 12          XGBoost 0.4965245 0.4146767 0.4194652 0.5053933
##           AUC
## 1  0.7552616
## 2  0.7359076
## 3  0.7795353
## 4  0.7578509
## 5  0.7765284
```

```
## 6 0.7861685
## 7 0.7861773
## 8 0.7781371
## 9 0.7734690
## 10 0.7851856
## 11 0.7786374
## 12 0.7683769
```

```
model_results_combined <- model_results
```